



第二课

人工智能基础

复旦大学可信具身智能研究院

陈智能

2026-09-15



课程概要

从机器学习到深度学习

创建第一个深度神经网络

卷积神经网络基本结构

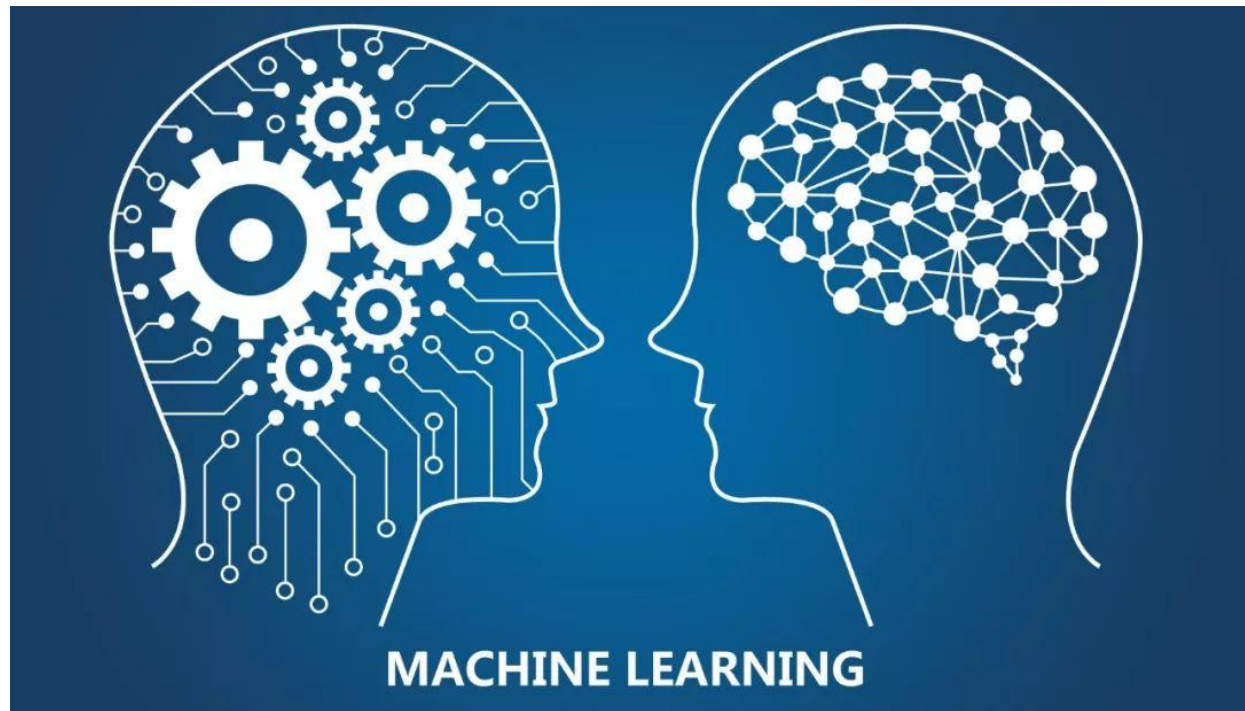
Transformer基础

Part 1

从机器学习到深度学习

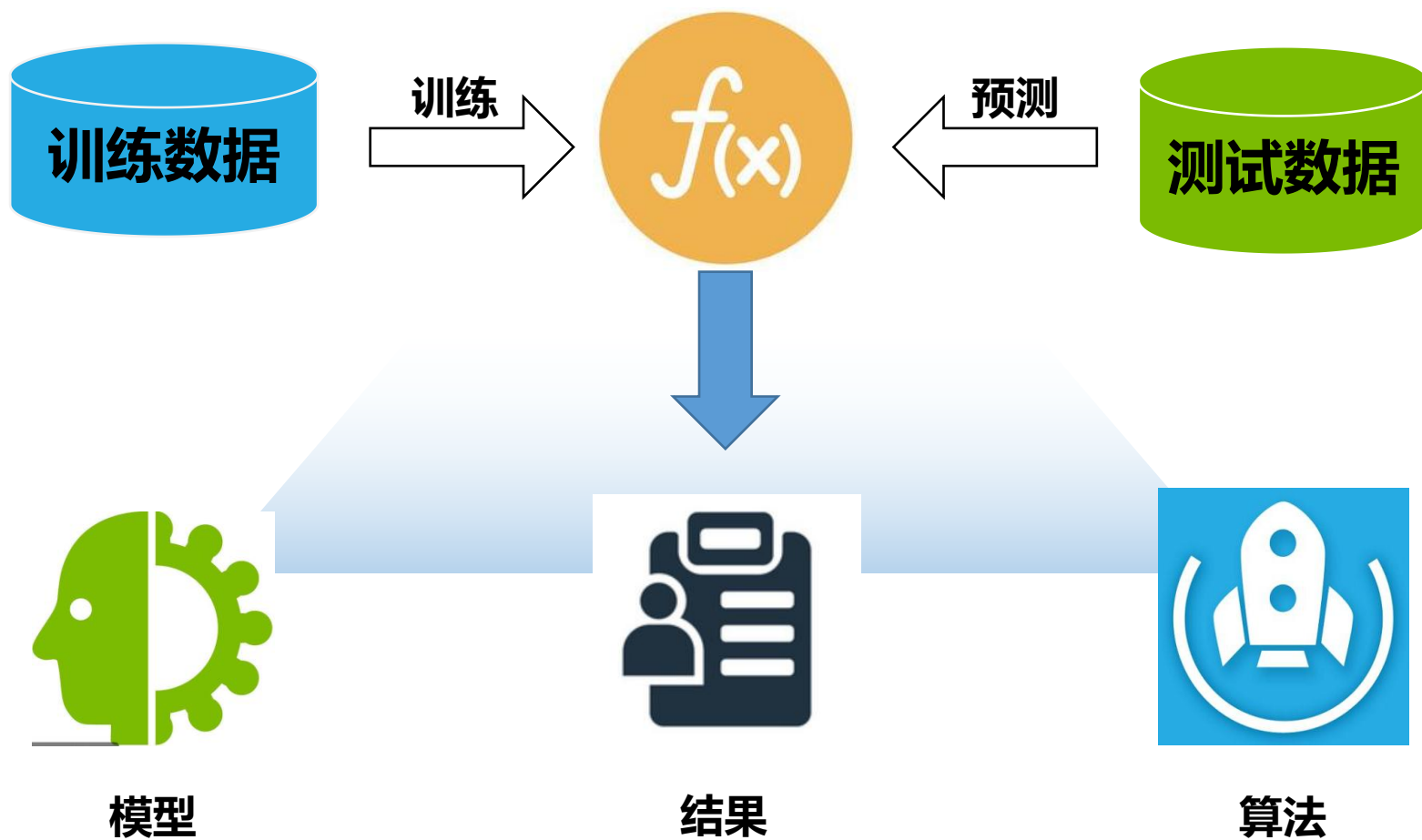
什么是机器学习

机器学习 (Machine Learning) 是一门多领域**交叉学科**，**专门研究计算机怎样模拟或实现人类的学习行为**，以获取新知识或技能，重新组织已有的知识结构使之不断改善自身的性能





什么是机器学习



机器学习:

经验Experience (E)、任务Task (T)、任务完成效果的衡量指标Performance measure (P)。有了这三个概念，机器学习的定义可以表述为：在有了经验E的帮助后，机器完成任务T的衡量指标P会变得更好

机器学习特点

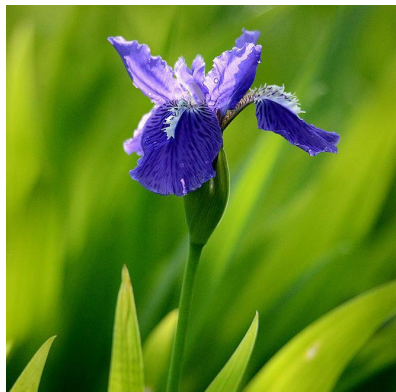
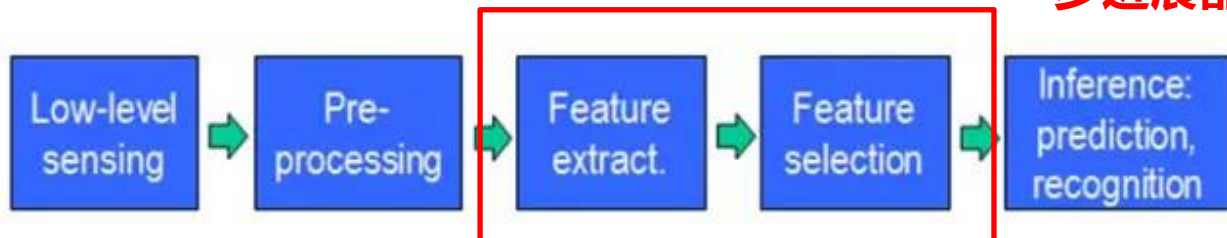
测量空间 → 特征空间 → 类别空间

核心任务

天花板已现，每一步进展都极为不易

传统方法

➤ 人工特征提取+分类器



1. 收集并标注几百张鸢尾花的图片
2. 观察不同的鸢尾花图片，并绞尽脑汁选择或设计一些特征形状，颜色，纹理；
SIFT, HOG, Gabor, LBP, Haar...
3. 根据设计的特征，提取图像的特征，并选择分类器进行训练和测试
4. 结果不好的话，回到第二步



传统机器学习 V.S. 深度学习

测量空间 $\longrightarrow$ 特征空间 $\longrightarrow$ 类别空间

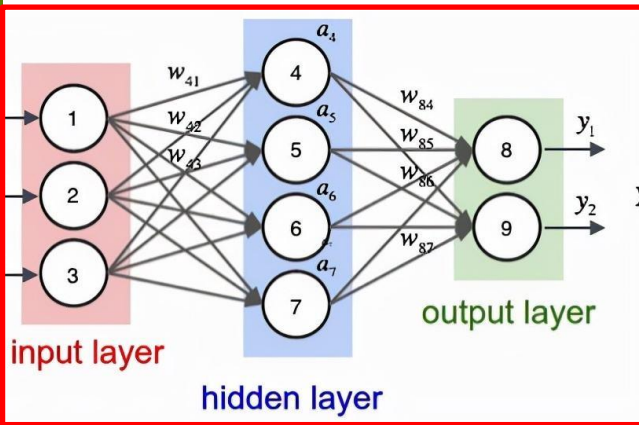
核心任务

深度学习方法

- 提供图像和标签，通过网络自己去学习特征



深度学习带来了视觉计算等领域
研究范式的重大变革

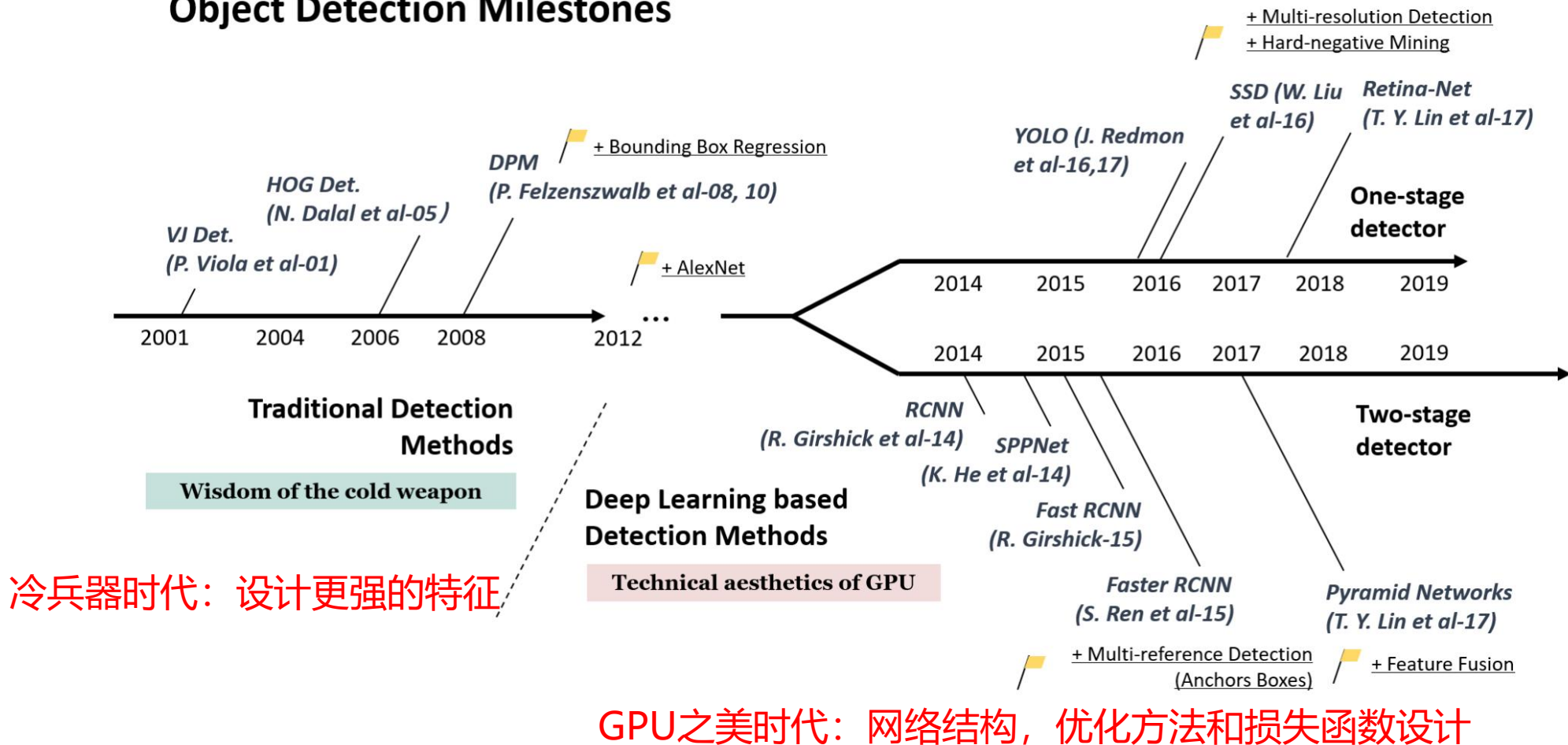


1. 收集并标注几千张图像
2. 确定网络结构、优化方法等，交给机器绞尽脑汁完成任务



深度学习带来研究范式变革

Object Detection Milestones



Part 2

创建第一个深度神经网络



神经网络引入

深度神经网络涉及的数学极其普通，每一位同学都可以搞定

人工智能基本上是一台猜测机器，是一个可以让你输入信息的软件，实现方式在数学上是相当普通的



如果你真正了解人工智能系统运行的具体细节，你会发现其中有些数学，但实际上在大多数情况下并不是最先进的数学

实际上，它的实现方式在数学上是相当普通的，只需接受你的输入，把它分成小部分，将每个单词或任何东西编码为数字，将这些数字乘以权重，然后将它们结合起来，也许会先截断它们，然后再将它们乘以权重并将它们结合起来，这样做几百次左右，就会得到答案

人工智能与数学

陶哲轩2024年8月在IMO2024（牛津大学）的演讲

<https://www.bilibili.com/video/BV1XE4m1R7Hp/>



创建第一个深度神经网络

神经网络的目标：寻找一个合适的函数

学习任务：寻找手写识别函数，候选函数集合为 $f, g, h \dots$

$$f\left(\boxed{\text{人}}\right) = \text{"人"}$$

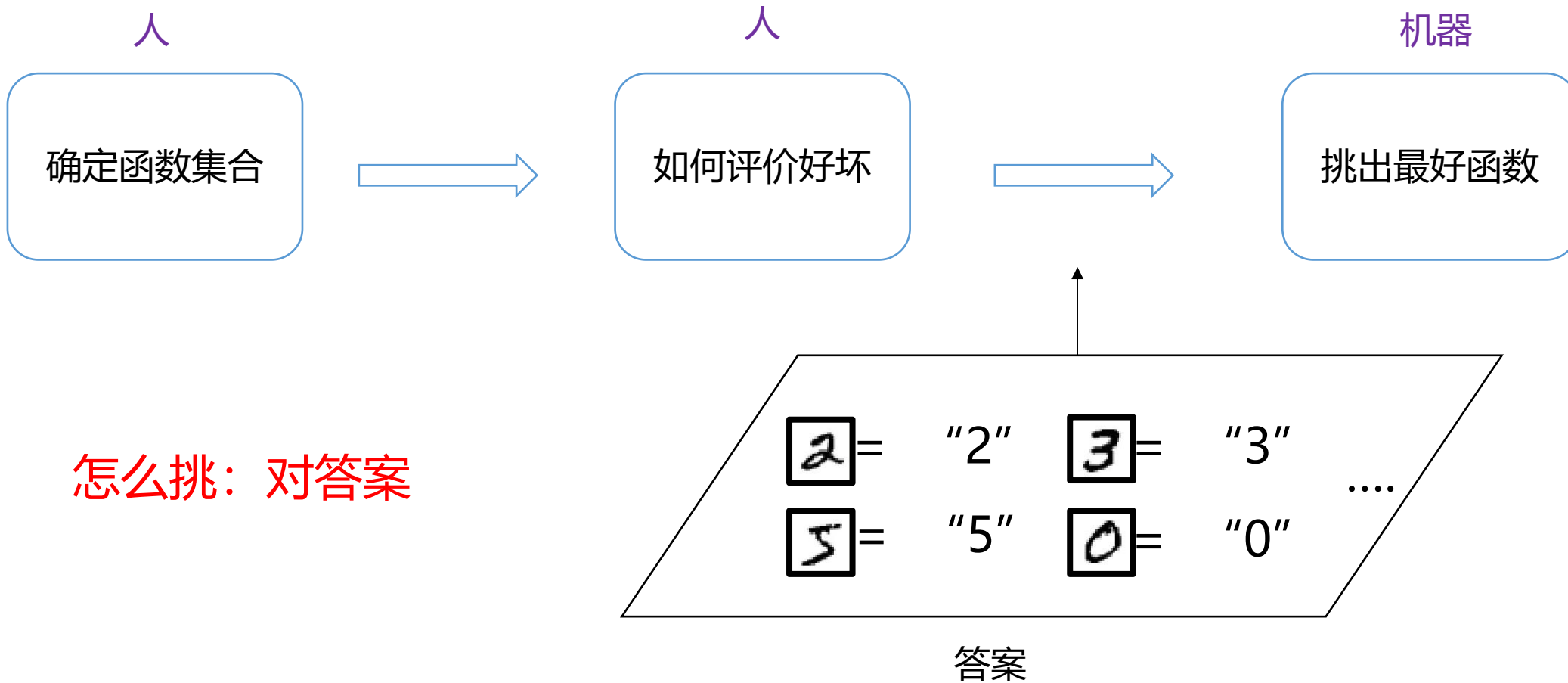
$$g\left(\boxed{\text{人}}\right) = \text{"8"}$$

$$h\left(\boxed{\text{人}}\right) = \text{"2"}$$



创建第一个深度神经网络

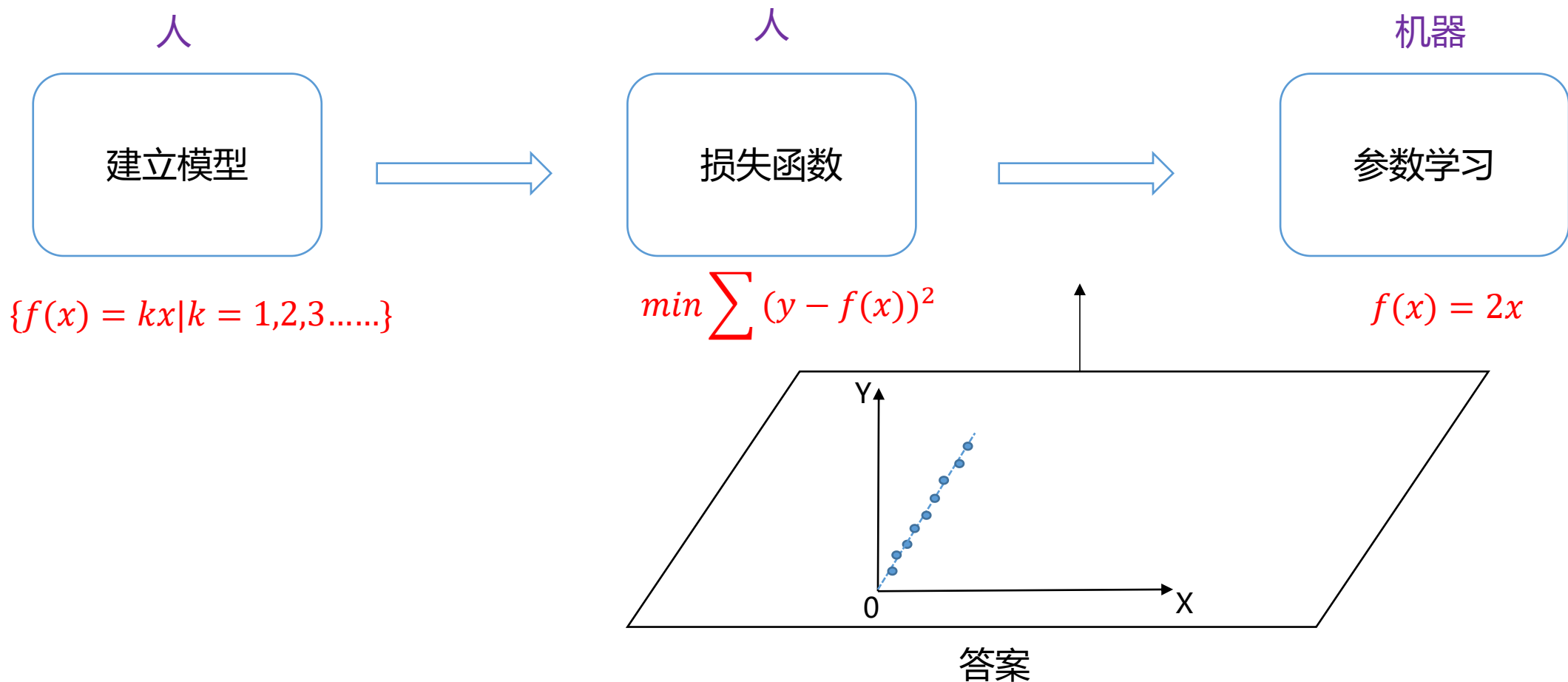
神经网络的目标：寻找一个合适的函数





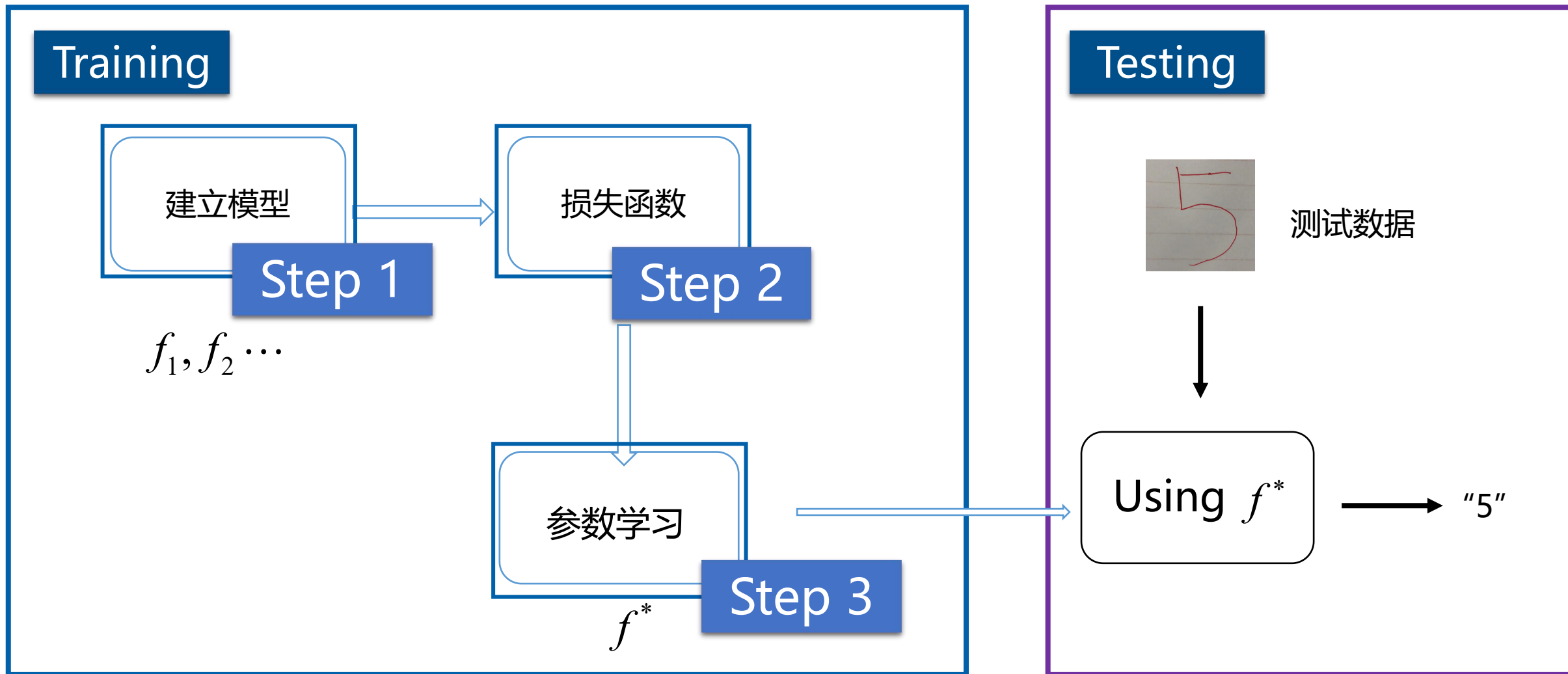
创建第一个深度神经网络

神经网络的目标：寻找一个合适的函数



创建第一个深度神经网络

整体流程





神经网络学习步骤

建立模型

- 选择什么样的网络结构
- 选择多少层数，每层选择多少神经元

损失函数

- 选择常用损失函数，平方误差，交叉熵，...

参数学习

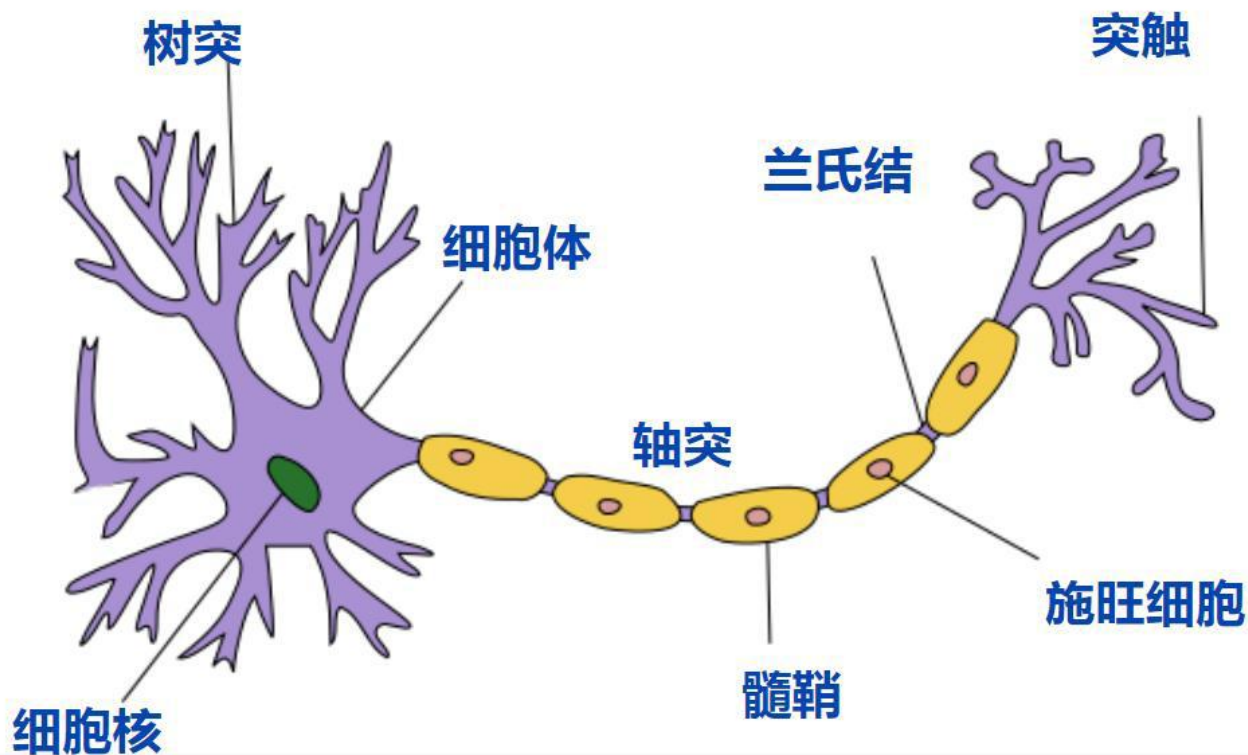
- 梯度下降
- 反向传播算法

建立模型

来自生物神经元的灵感

- 每个神经元都是一个多输入单输出的信息处理单元
- 神经元具有空间整合和时间整合特性
- 神经元输入分兴奋性输入和抑制性输入两种类型
- 神经元具有阈值特性

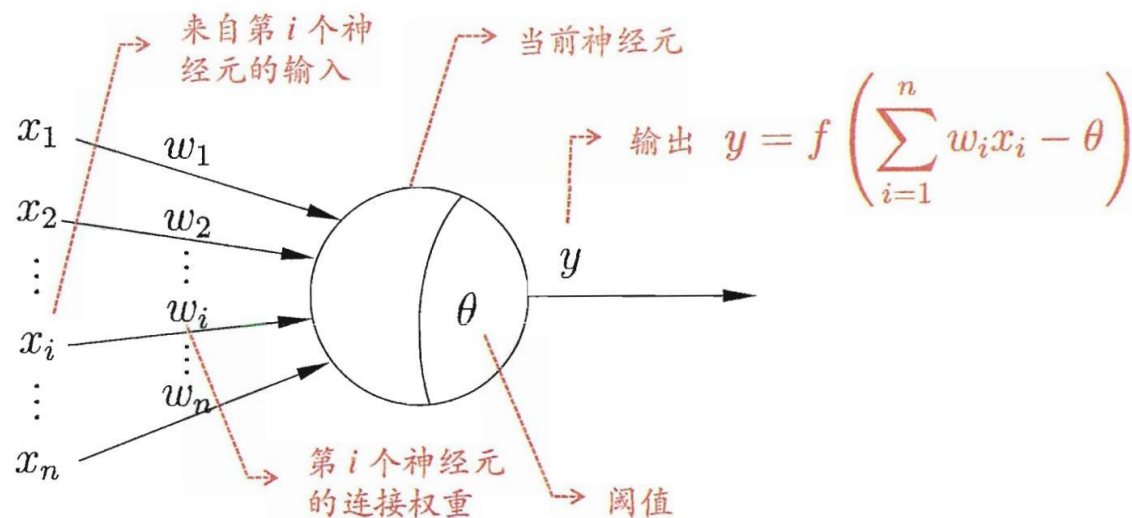
不清楚怎么做，那就参考神经科学



建立模型

来自生物神经元的灵感

- 每个神经元都是一个多输入单输出的信息处理单元;
- 神经元具有空间整合和时间整合特性;
- 神经元输入分兴奋性输入和抑制性输入两种类型;
- 神经元具有阈值特性。



多输入信号进行累加 $\sum_i x_i$

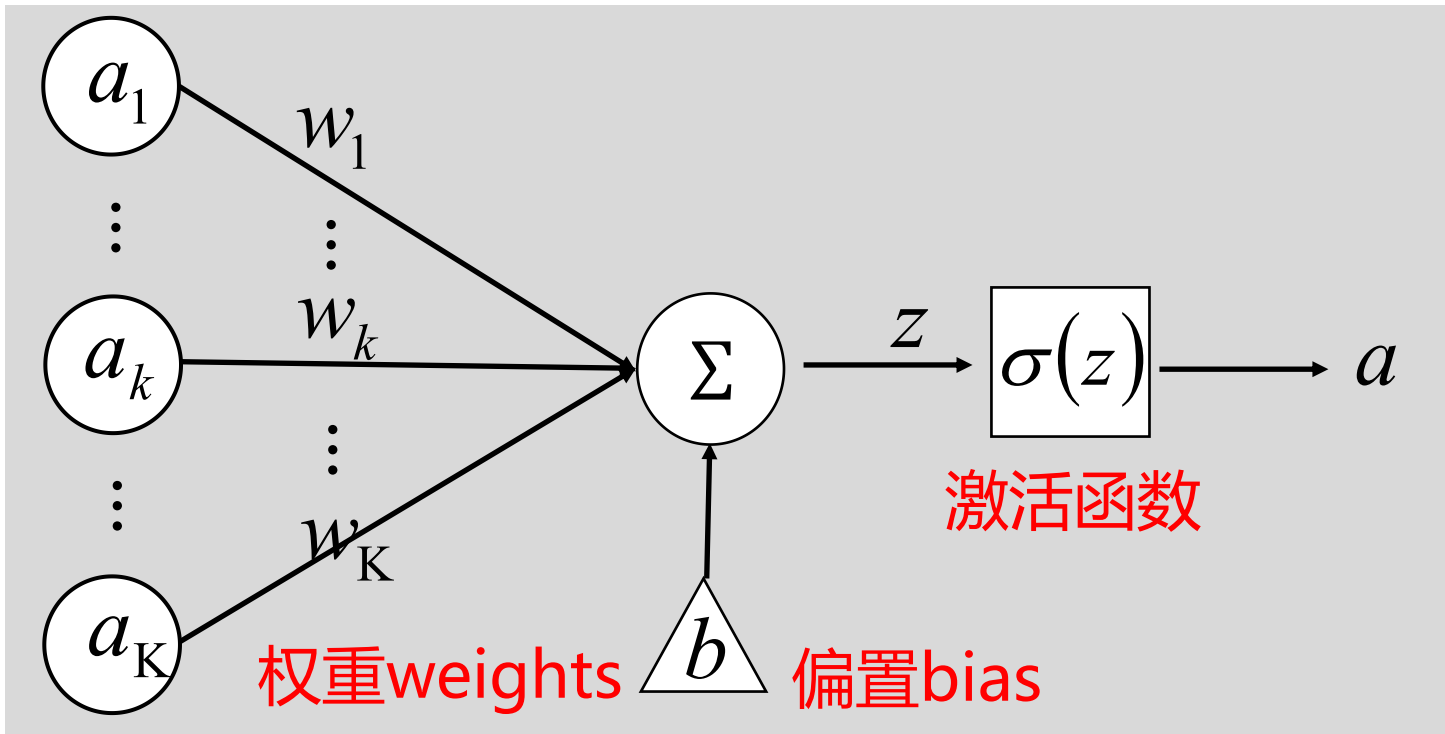
权值 w_i 正负模拟兴奋/抑制，大小模拟强度

输入和超过阈值 θ ，神经元被激活

建立模型

神经网络实现

$$z = a_1 w_1 + \cdots + a_k w_k + \cdots + a_K w_K + b$$



理解：简单线性函数

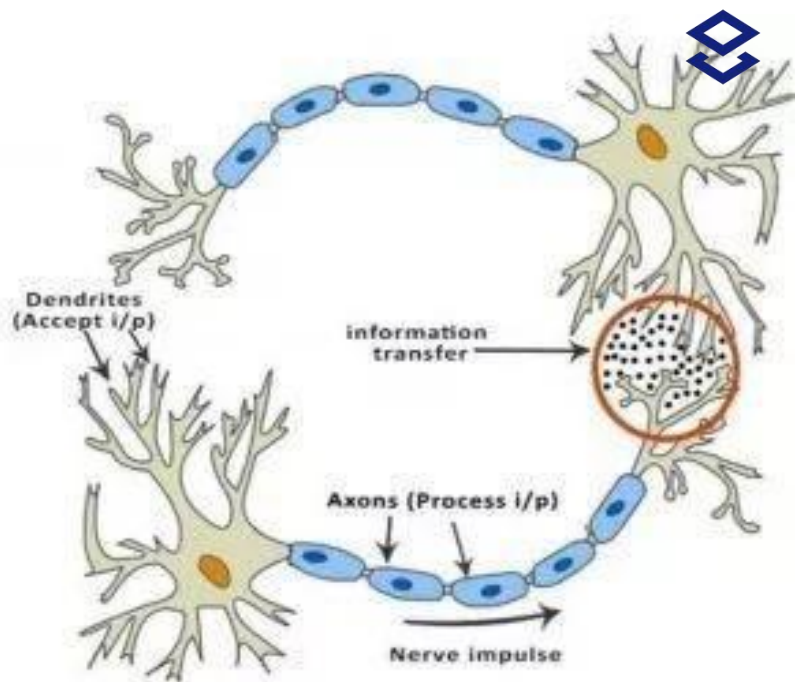
$$f(\mathbf{x}) = \mathbf{k}\mathbf{x} + b$$

(k 是斜率, b 是截距)

激活函数

◆ 神经元节点间的激活

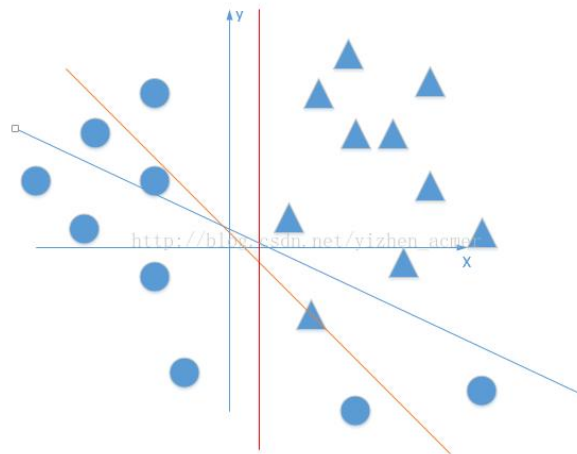
神经元继续传递信息、产生新连接的概率
(超过阈值被激活，但不一定传递)



◆ 没有激活函数相当于矩阵相乘

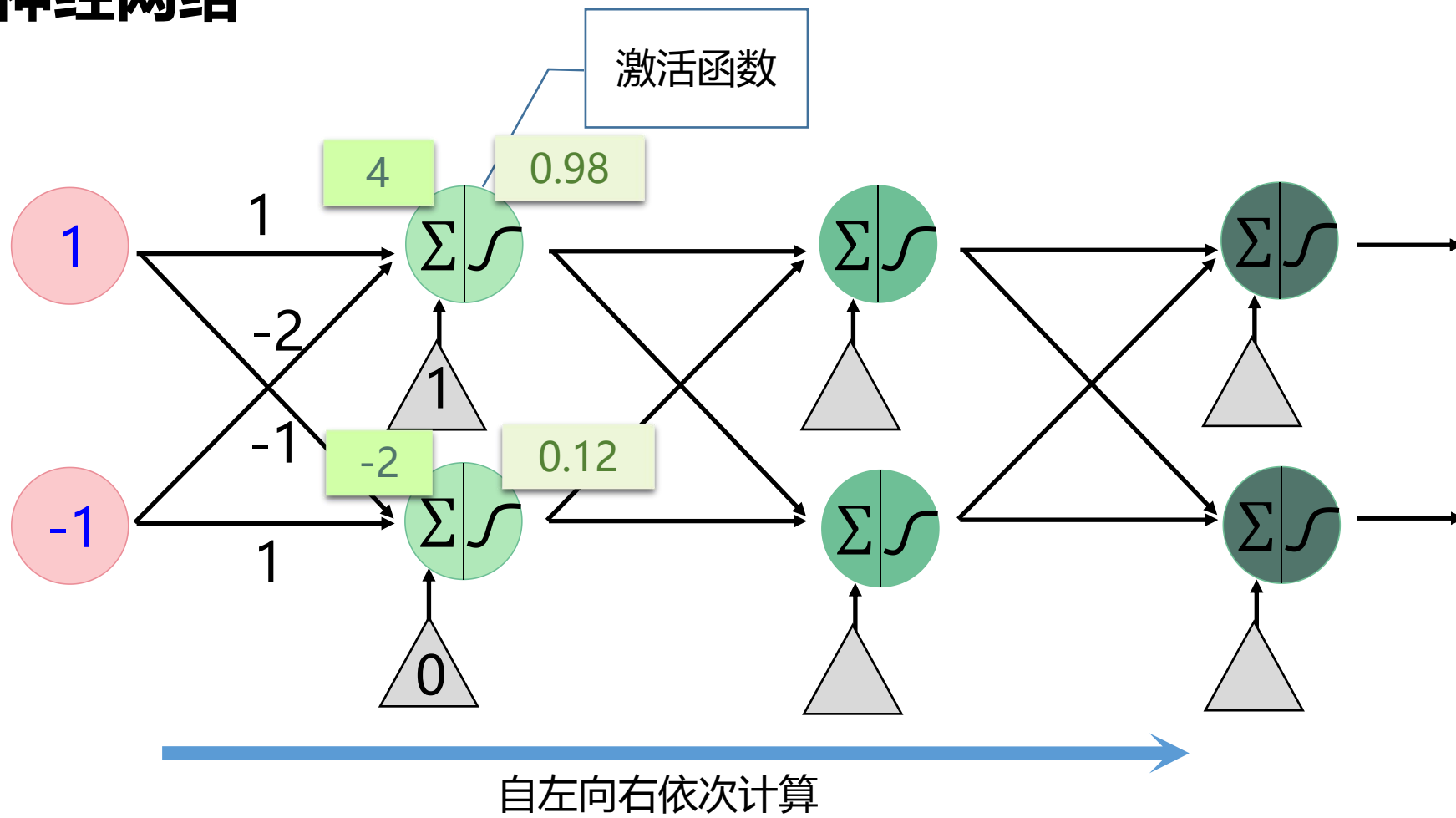
- 多层和一层一样
- 只能拟合线性函数

$$x^T W_1 \cdots W_n = x^T \prod_{k=1}^n W_k$$



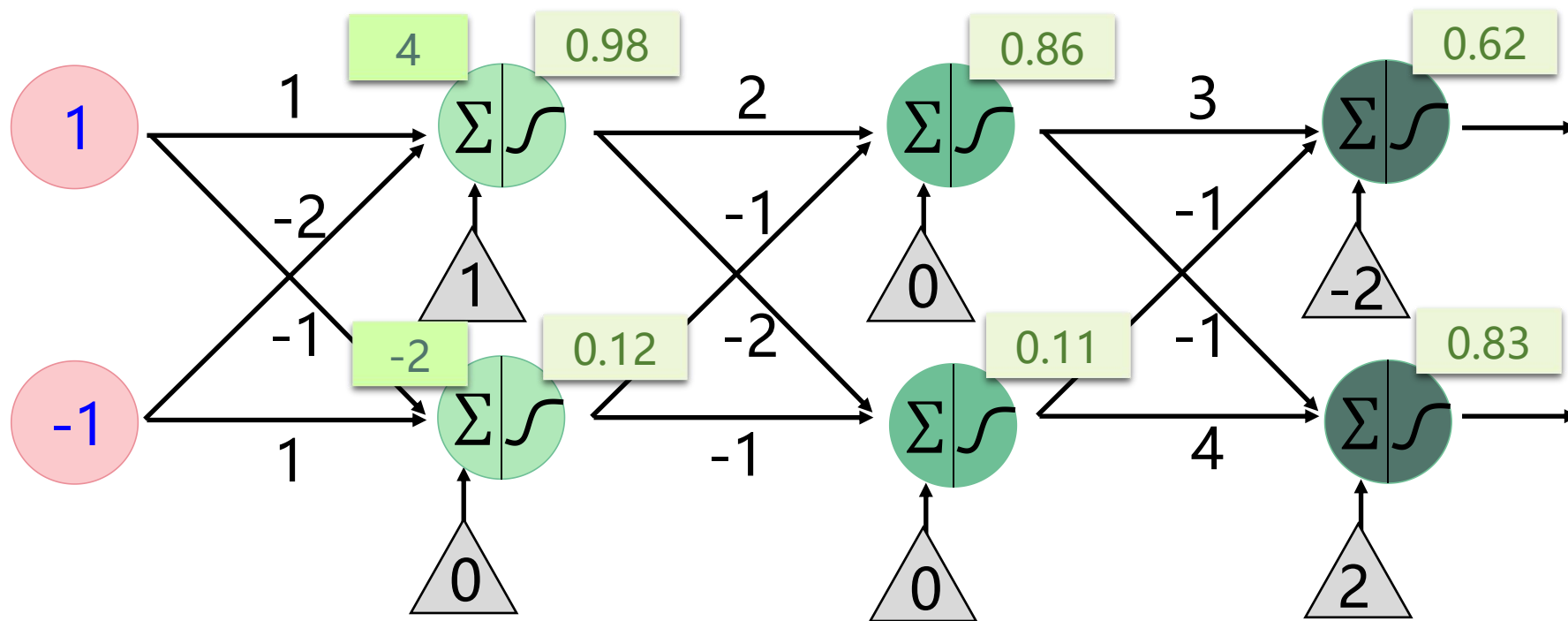
建立模型

前馈神经网络



建立模型

前馈神经网络

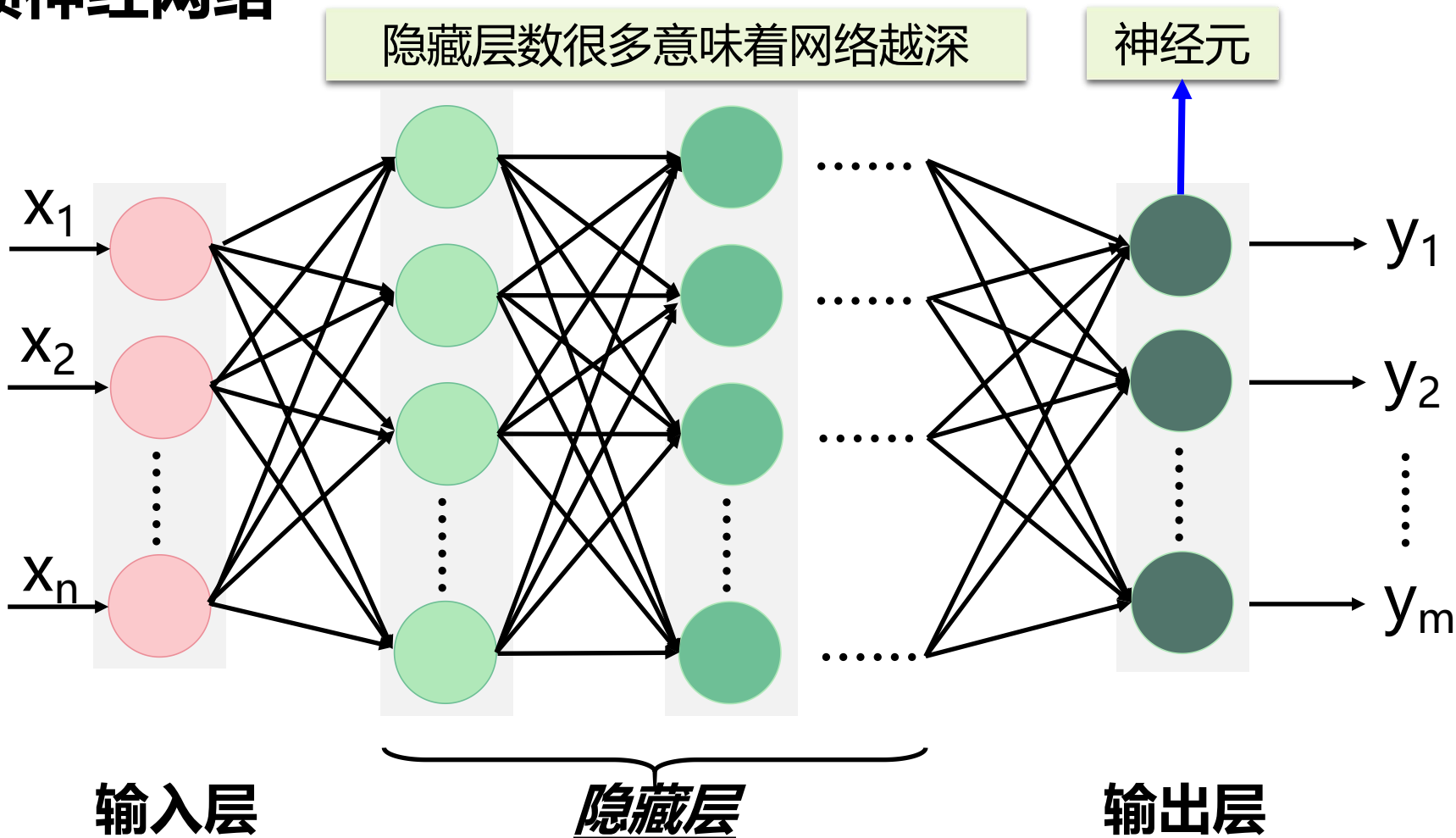


函数: $f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix}$

建立模型

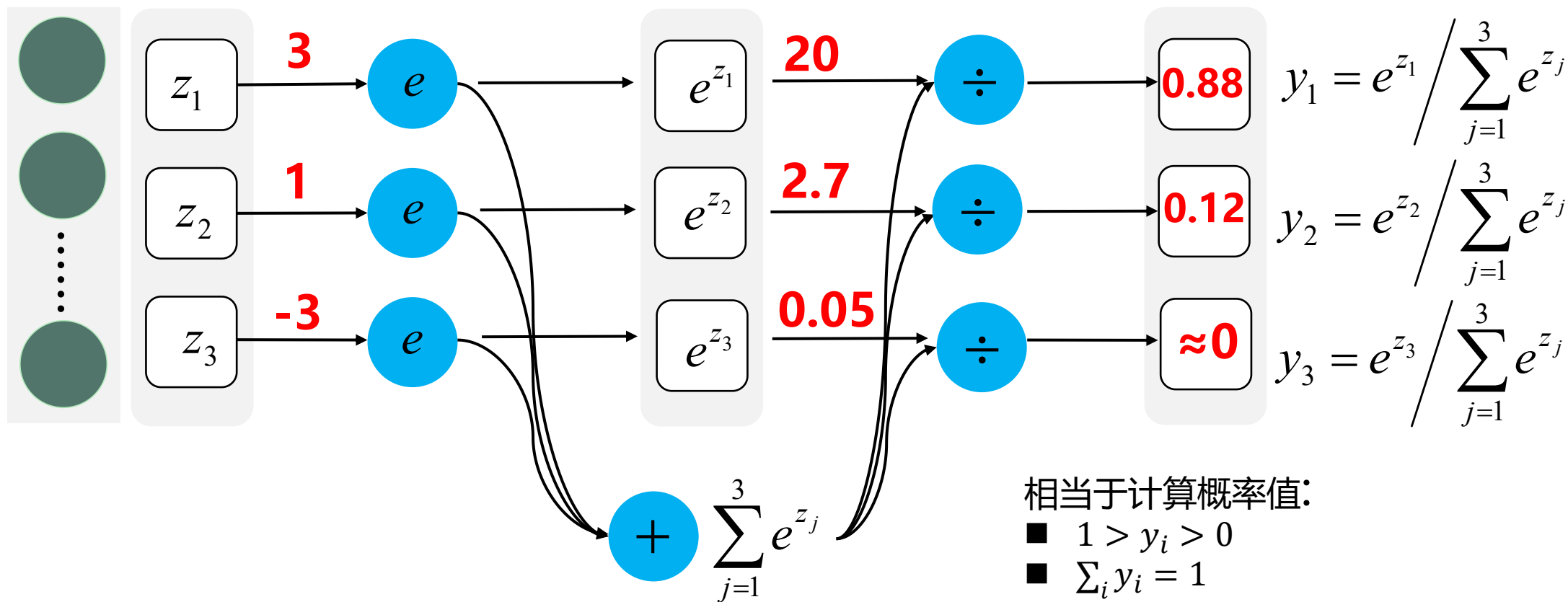
前馈神经网络

应该设置多少层？每一层多少个神经元？



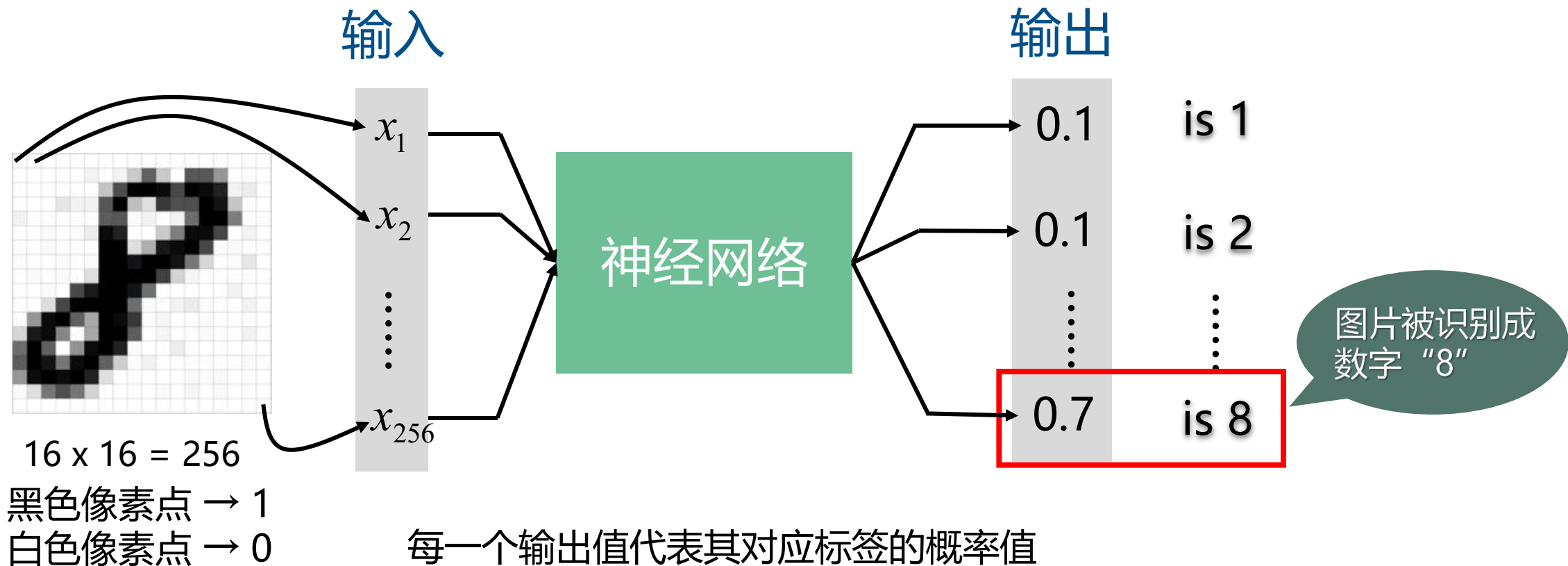
输出层

◆ 常用softmax函数作为输出层激活函数：容易理解、便于计算





应用示例：手写数字识别





神经网络学习步骤

建立模型

- 选择什么样的网络结构
- 选择多少层数，每层选择多少神经元

损失函数

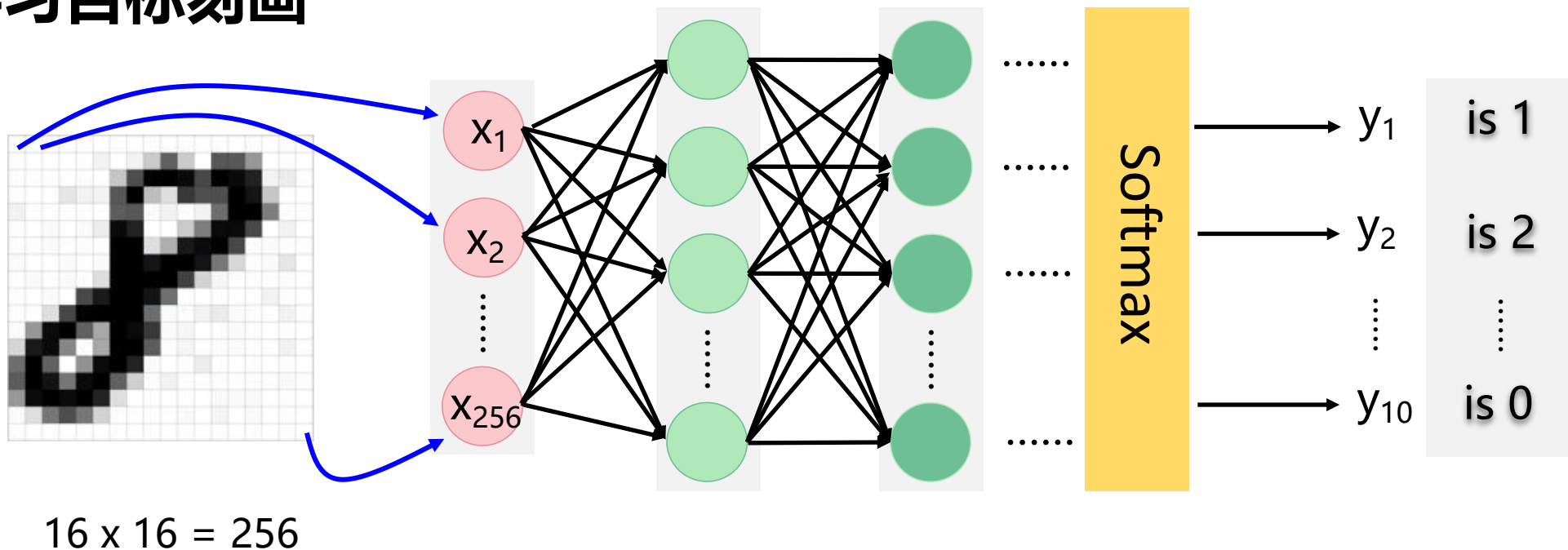
- 选择常用损失函数，平方误差，交叉熵，...

参数学习

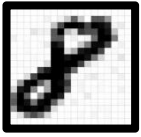
- 梯度下降
- 反向传播算法

损失函数

学习目标刻画

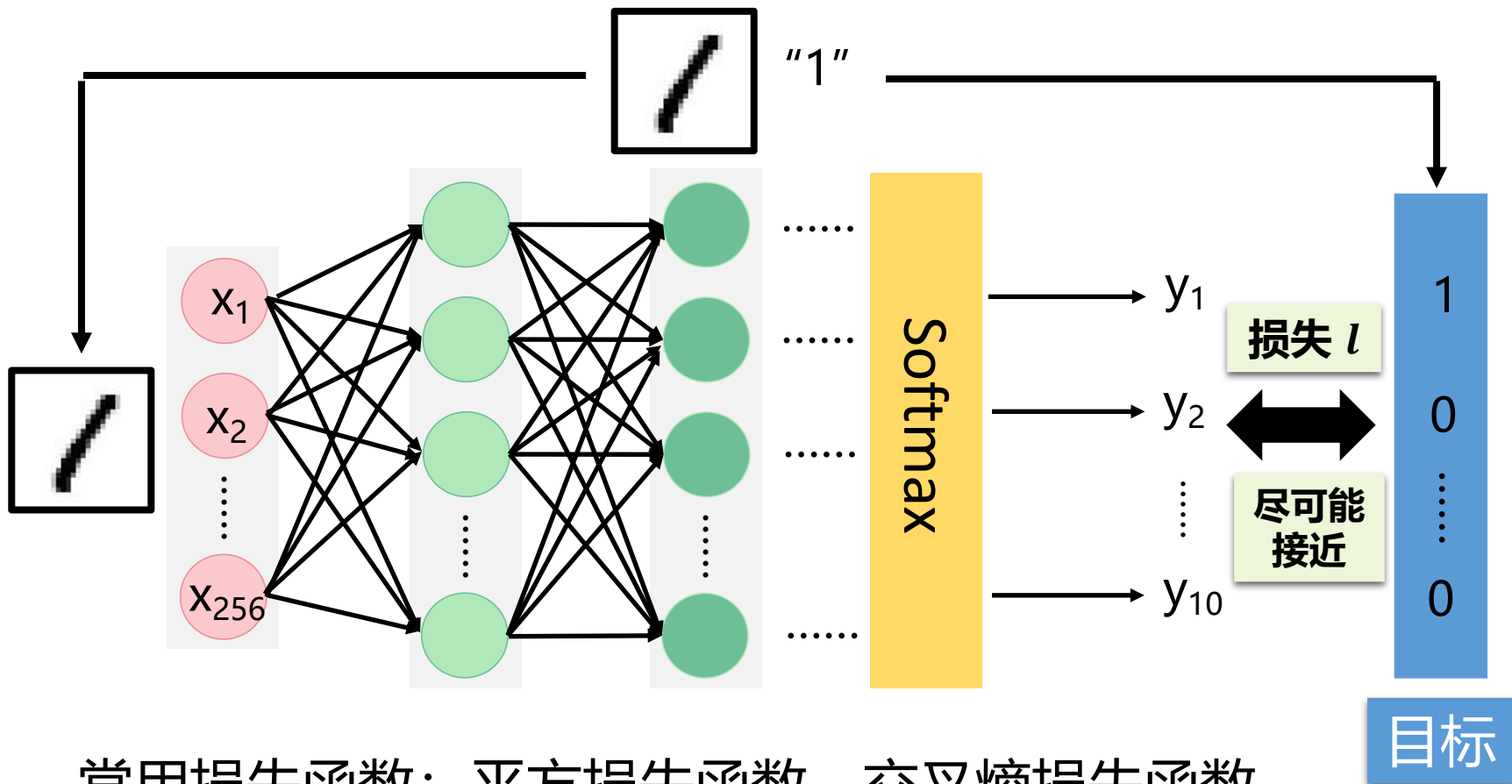


对于数字识别任务，设计用于分类的损失函数，使得学习目标变为：

输入：  $\rightarrow y_1$ 值最大； 输入：  $\rightarrow y_8$ 值最大；

学习目标刻画

好的参数使得所有训练数据的损失越小越好

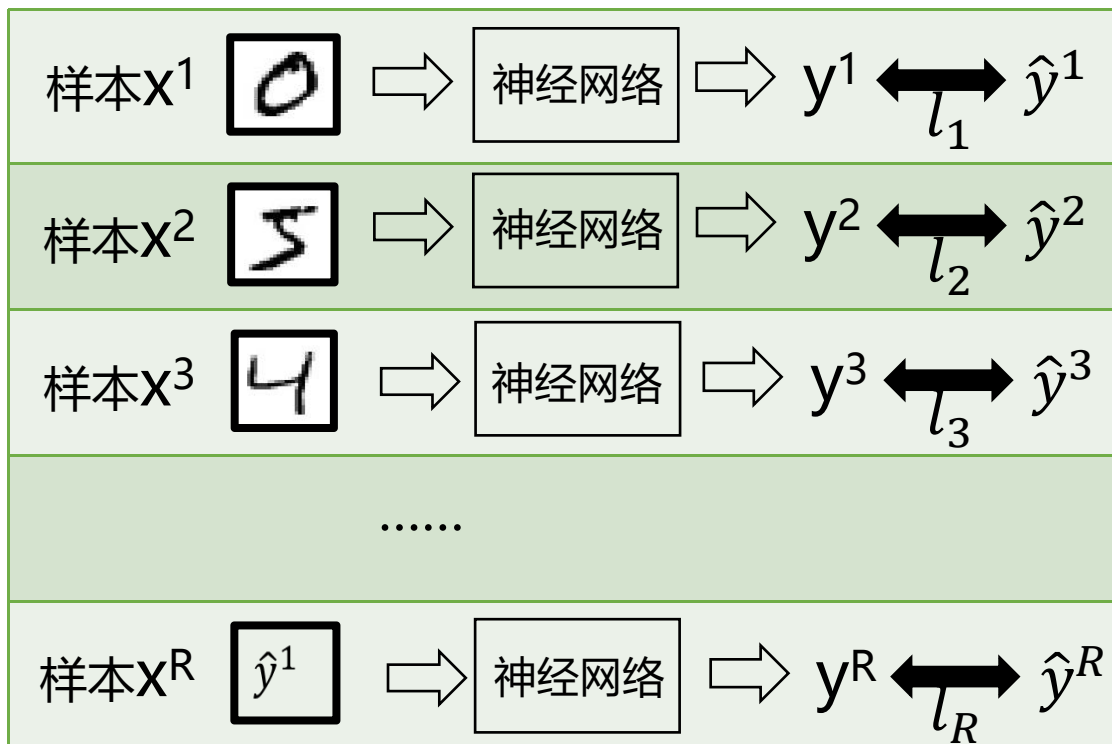


常用损失函数：平方损失函数、交叉熵损失函数

损失函数

损失函数

对所有R个训练数据:



总损失:

$$L = \sum_{r=1}^R l_r$$

尽可能小

找到一个函数使得总损失L最小

即确定参数使得总损失L最小



神经网络学习步骤

建立模型

- 选择什么样的网络结构
- 选择多少层数，每层选择多少神经元

损失函数

- 选择常用损失函数，平方误差，交叉熵，...

参数学习

- 梯度下降
- 反向传播算法

参数学习

❖ 参数量巨大，有限时间内的近似解是一个好选择

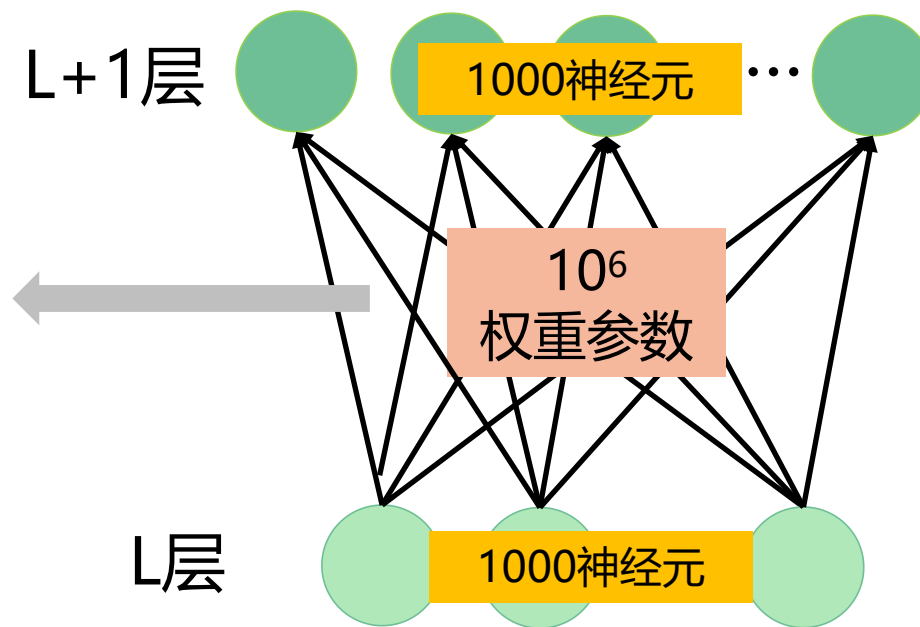
- 枚举所有可能的取值



网络参数 $\theta =$

~~$\{w_1, w_2, w_3, \dots, b_1, b_2, b_3, \dots\}$~~
参数个数巨大

- 寻找模型参数 θ^* 使得总损失 L 最小

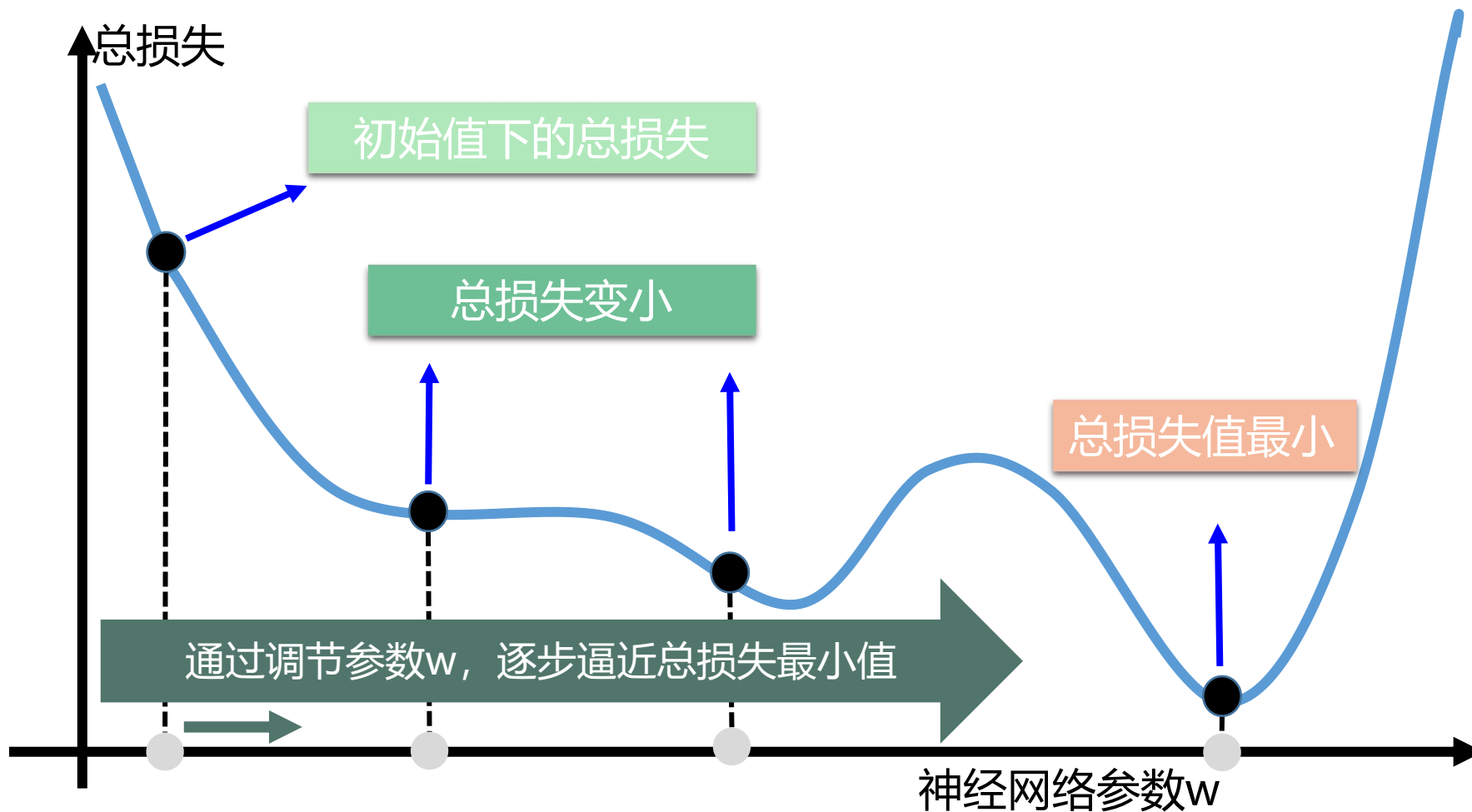


例如：手写数字识别模型有 8 层，每层 1000 神经元

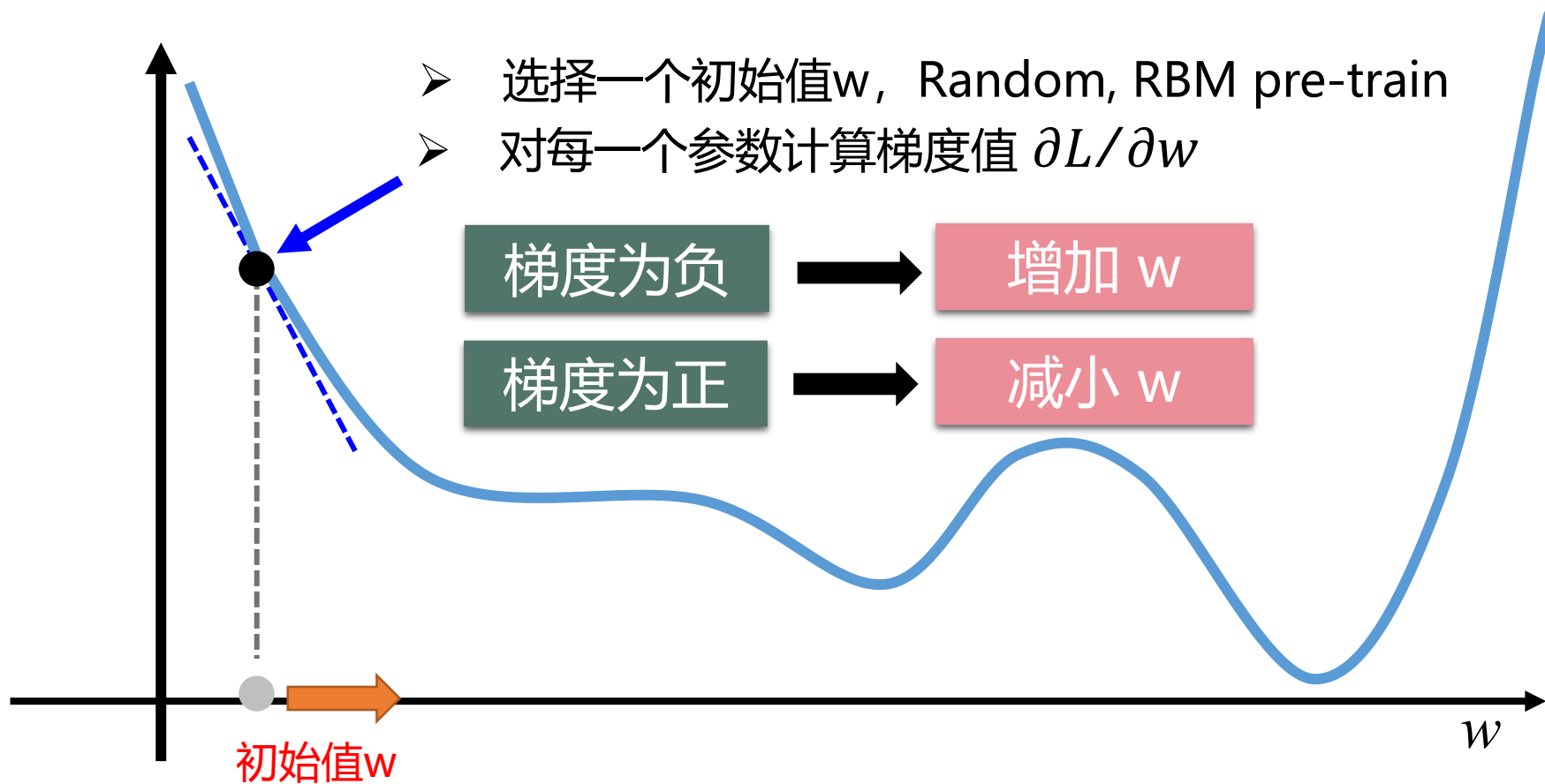


参数学习

梯度下降法



梯度下降法

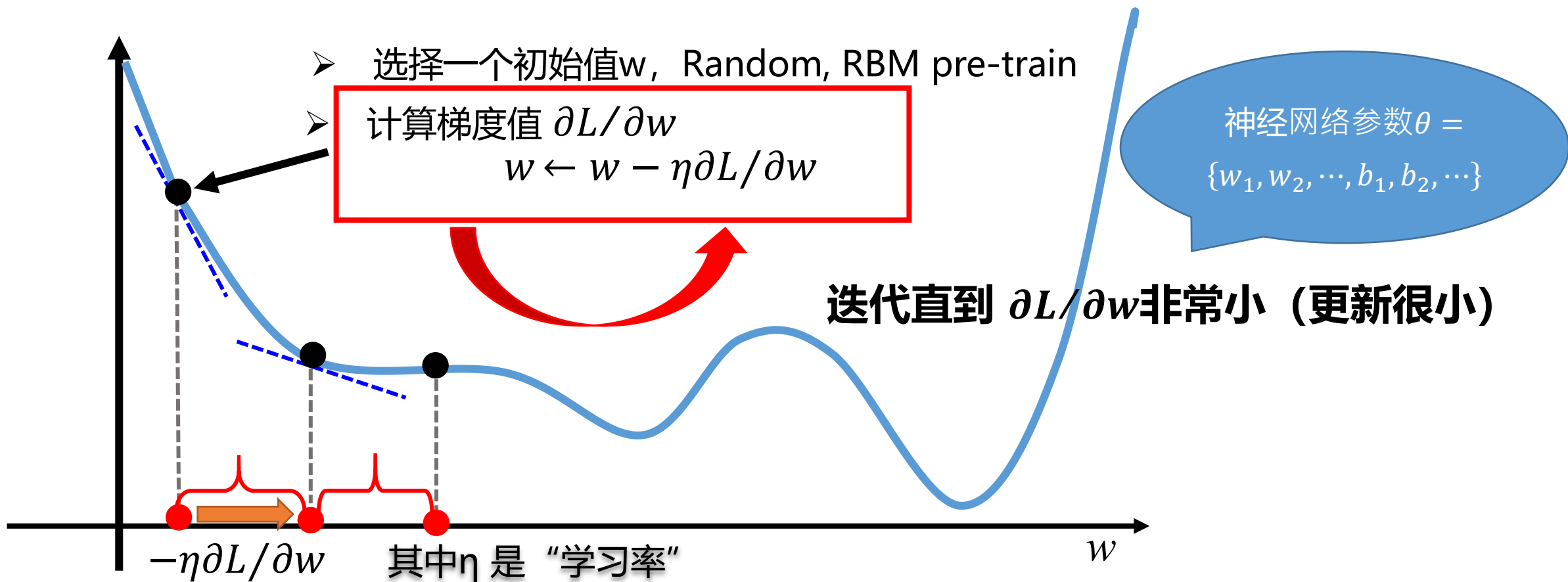


神经网络参数 $\theta = \{w_1, w_2, \dots, b_1, b_2, \dots\}$

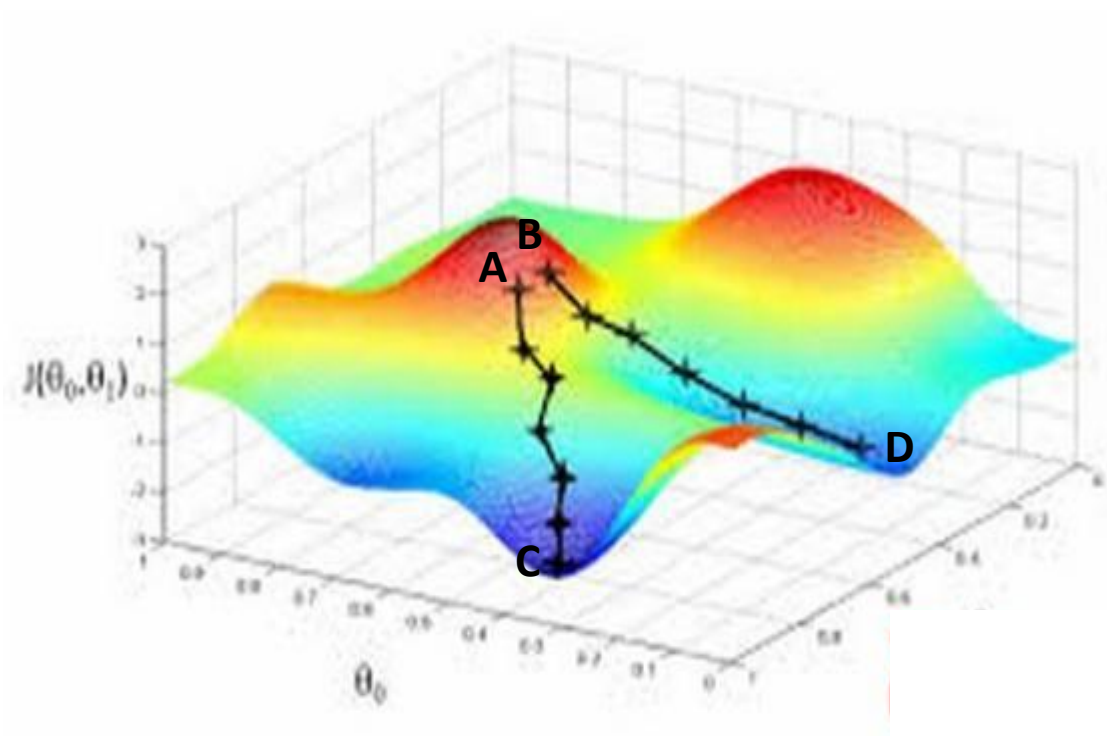


参数学习

梯度下降法



初始值影响



例如：

A $\longrightarrow$ C

B $\longrightarrow$ D

选取不同的初始值，可能到达不同的局部最小值



参数学习

反向传播算法

Hinton教授等给出的反向传播算法可以求解这个问题



只剩一个问题：
怎么求 梯度 $\partial L / \partial w$?

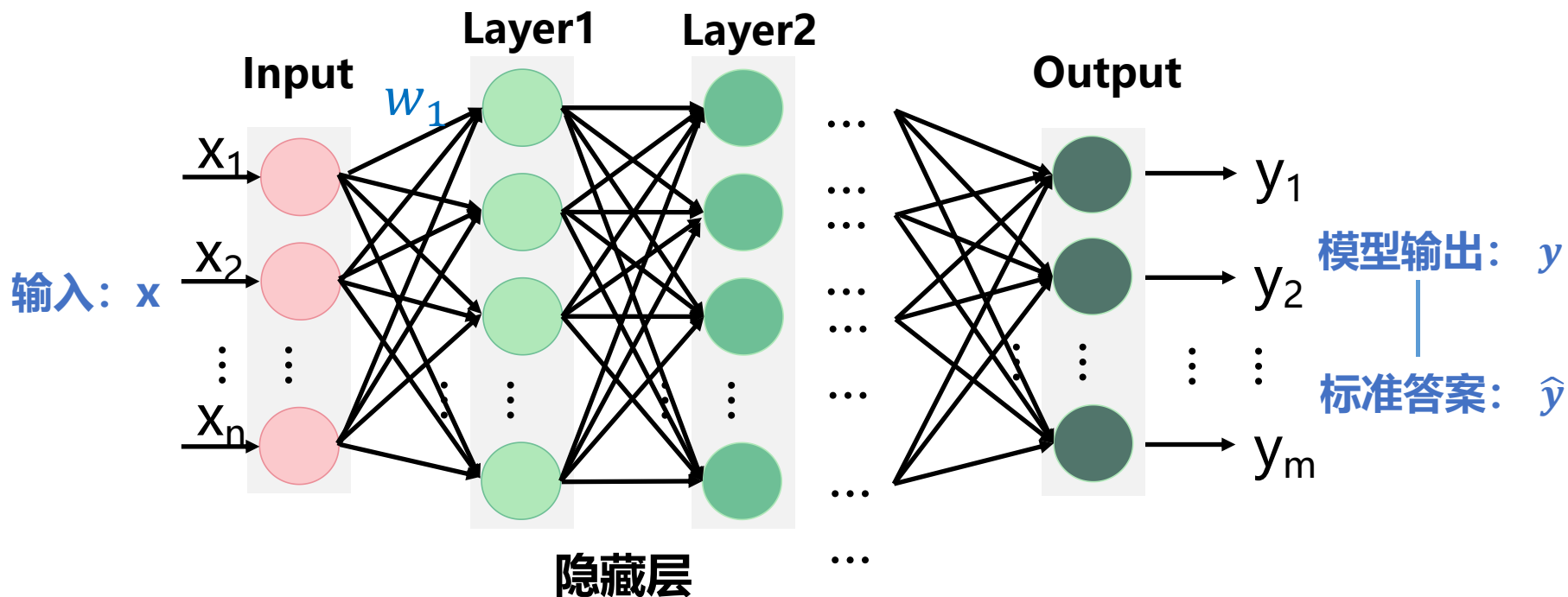
$$w_1 - \eta \partial L / \partial w_1$$

更新 w_1

计算 $\partial L / \partial w_1$

以此类推更新 $w_2 w_3 \dots$

总误差 L



Part 3

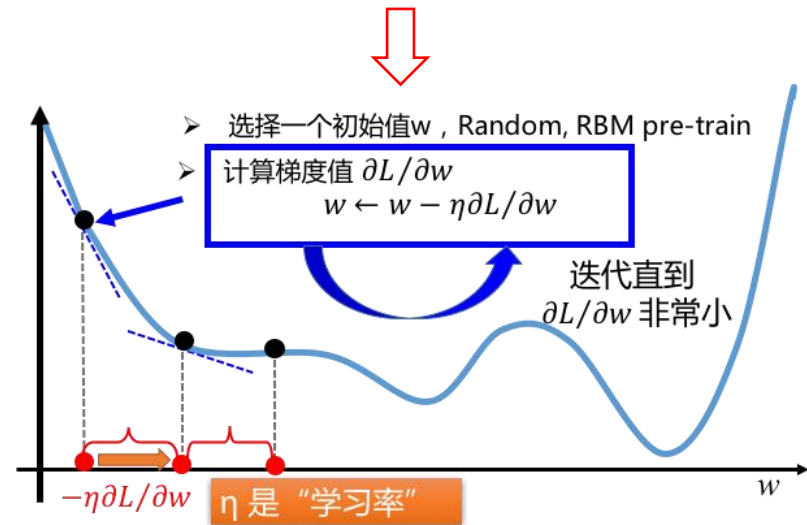
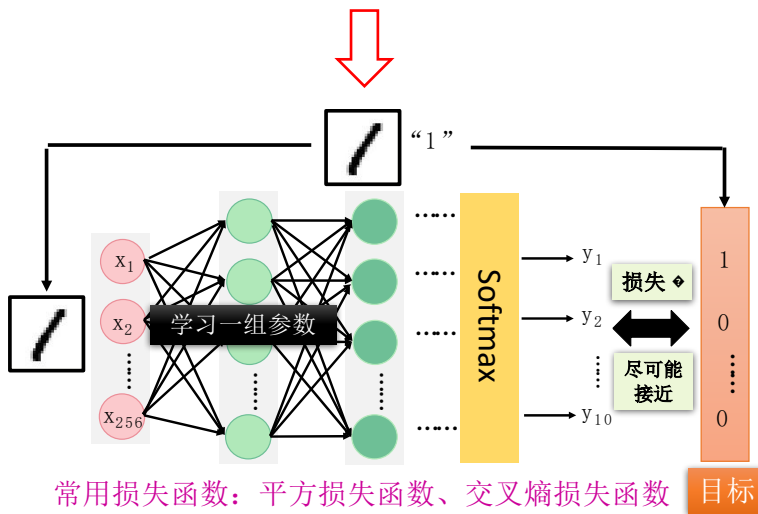
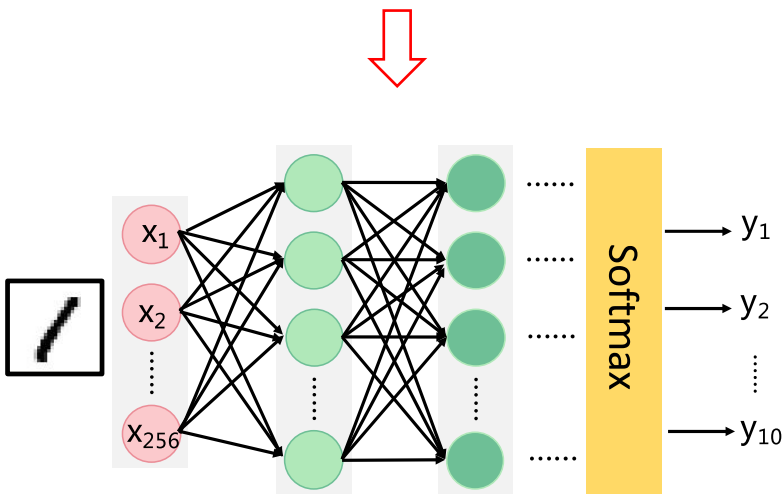
卷积神经网络

模型回顾

建立模型

损失函数

参数学习





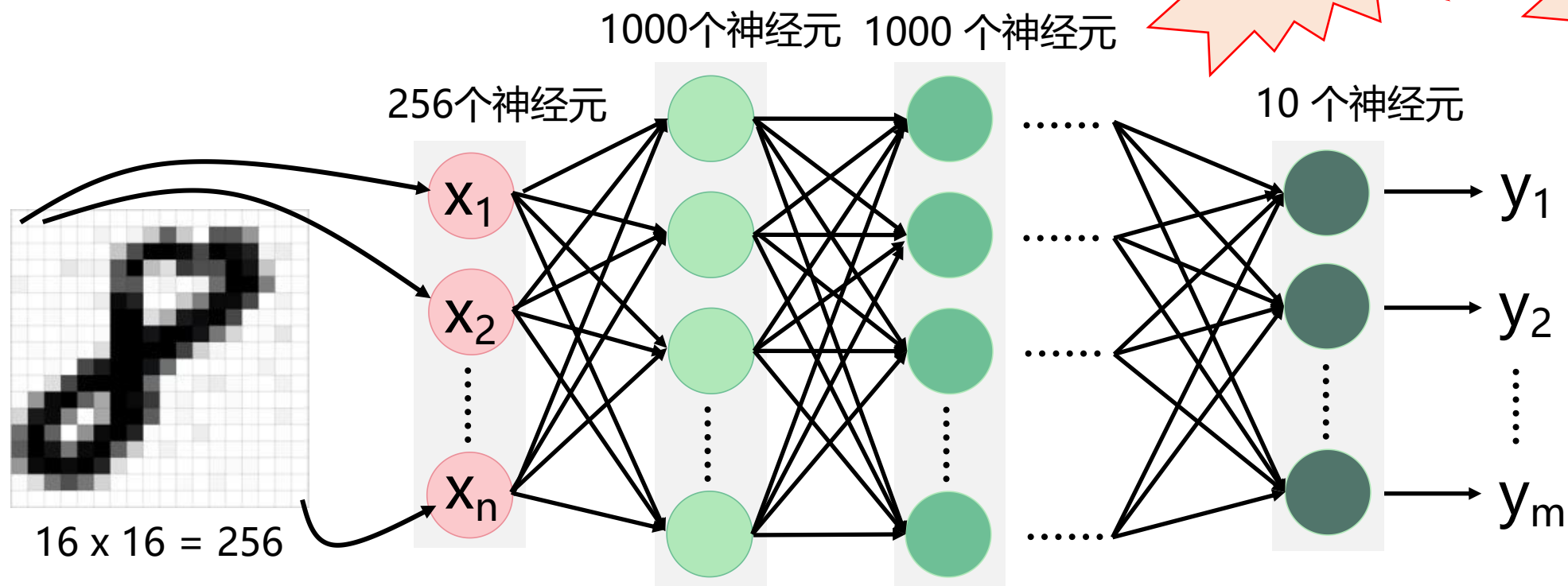
模型回顾

以上这种神经网络模型
存在哪些不足？



模型回顾

✧ 手写数字识别的例子

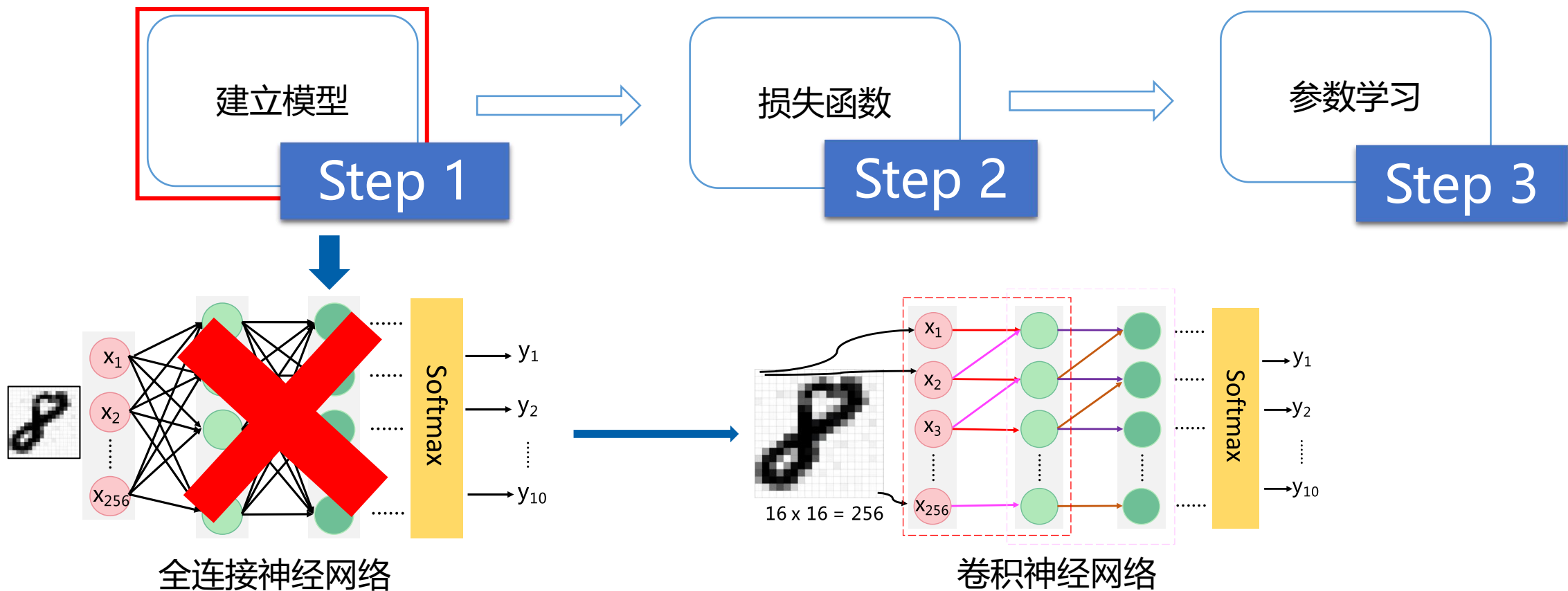


之前对16 x 16的图片进行手写数字分类任务

对100*100的图片做相同的任务：1) 网络结构需要调整，需要重新训练模型，调优可能涉及增加每层的神经元个数或者增加网络的层数；2) 参数量急剧增加

模型如何改进

如何解决前面的问题？





卷积神经网络

卷积神经网络结构上的三大特性

- ✓ 局部连接
- ✓ 权重共享
- ✓ 下采样

减少网络参数
加快训练速度
降低资源消耗

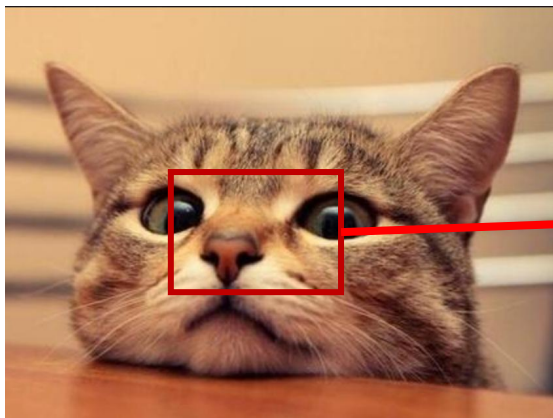
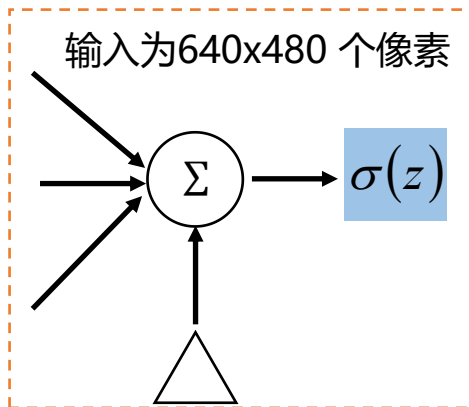


卷积神经网络

局部连接

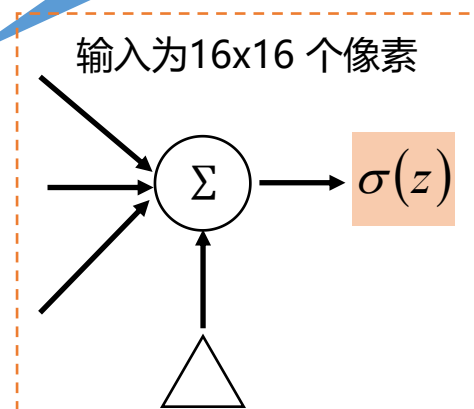


640x480



16x16

卷积神经网络

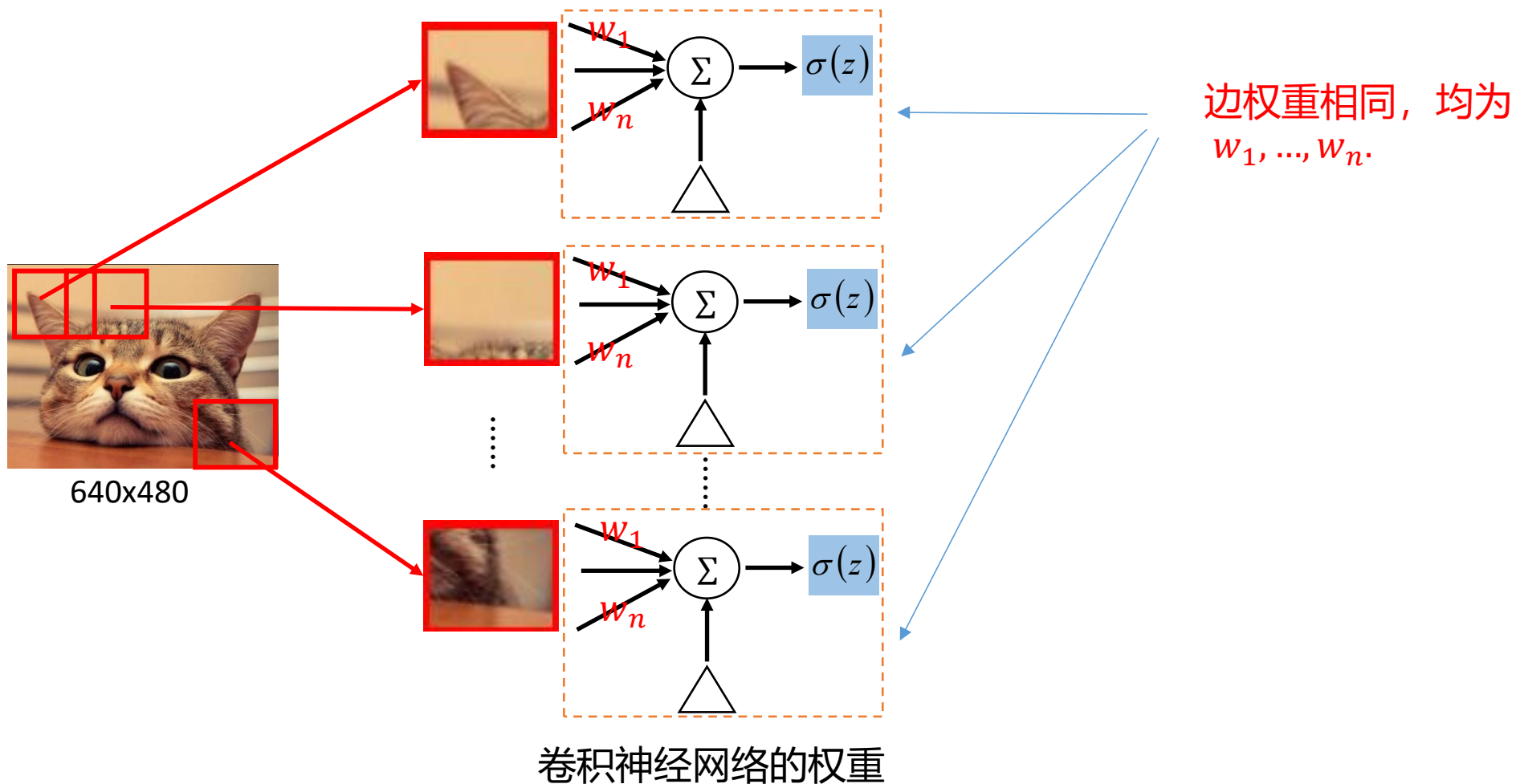


在进行图像识别的时候，
不需要对整个图像进行
处理，只需要关注图像
中某些特殊的区域



卷积神经网络

权值共享





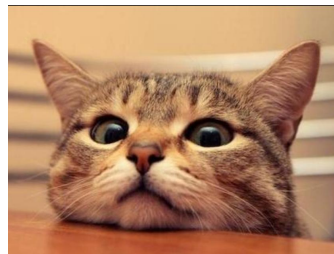
卷积神经网络

下采样



640x480

下采样
→

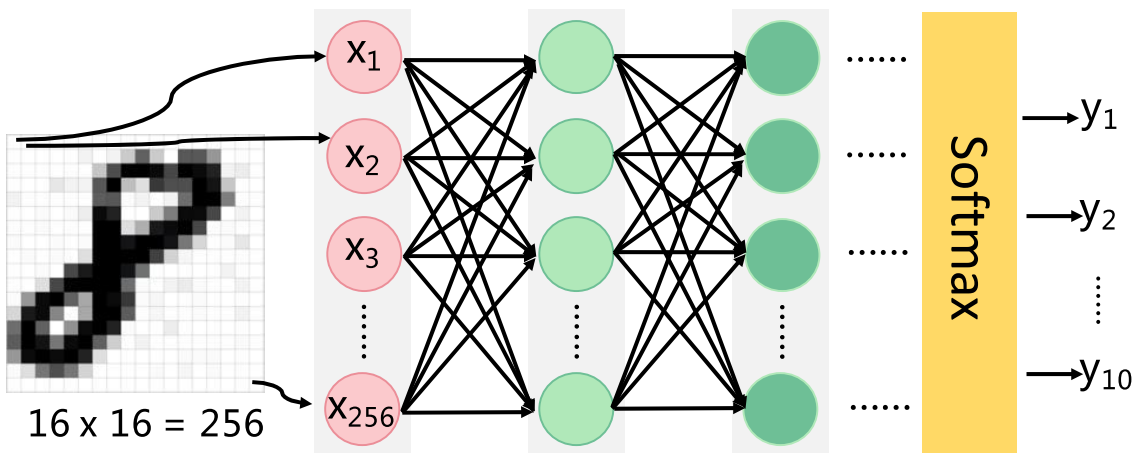


320x240

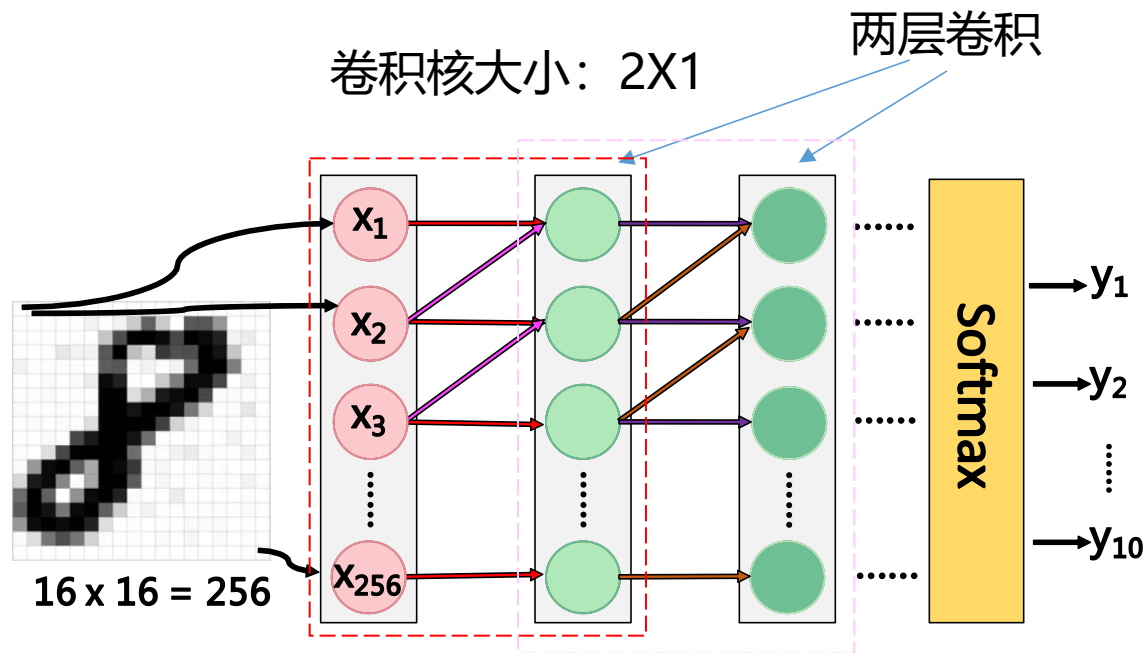
对图像像素进行下采样，得到小一些的图像

虽然下采样之后的图像尺寸变小了，但是同样的采样窗口，看到的图像区域就更大了

与全连接神经网络对比



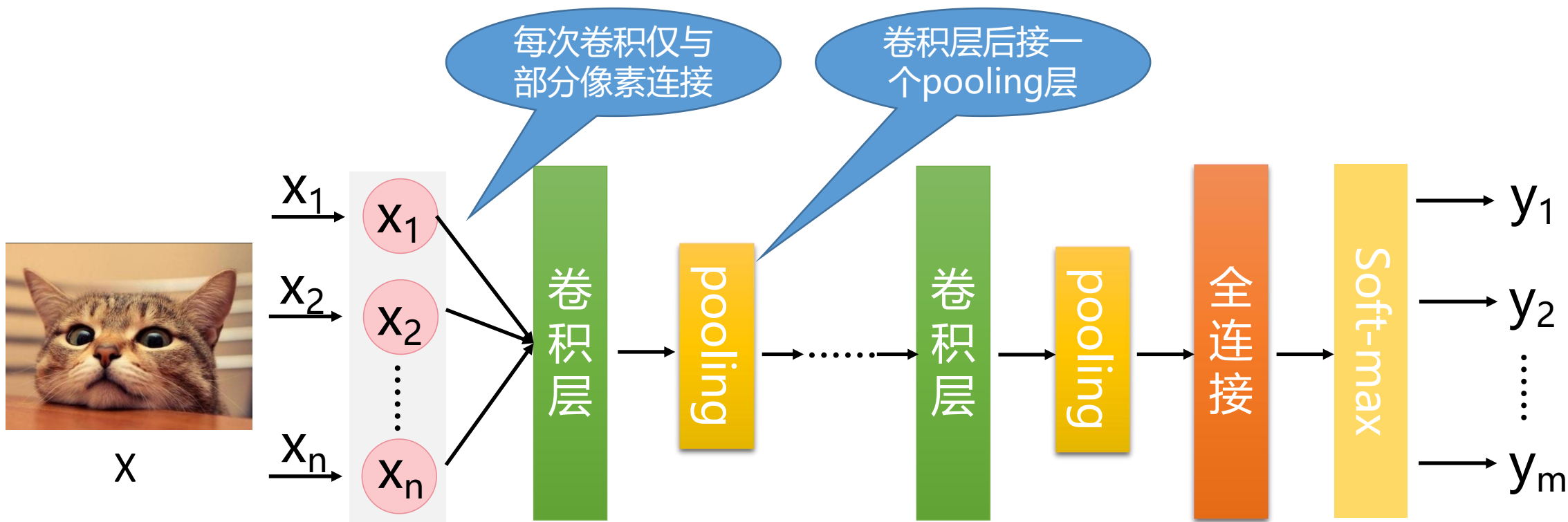
全连接神经网络



卷积神经网络

相同颜色的箭头代表相同的权重参数

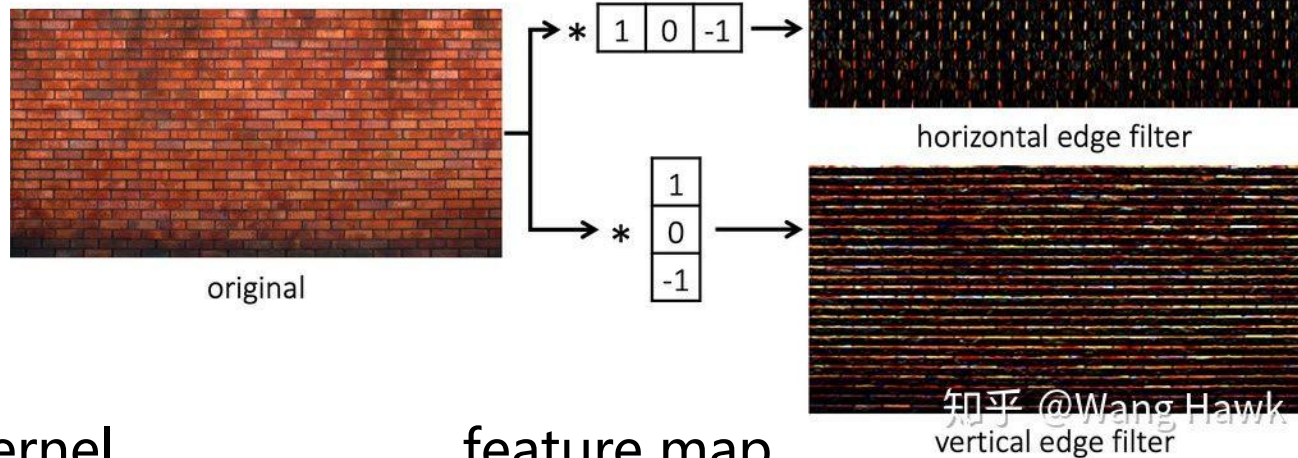
卷积神经网络基本结构





建立模型

卷积运算



input image
(输入图像)

10	10	10	10	10	10
10	10	10	10	10	10
10	10	10	10	10	10
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

kernel
(卷积核)

1	2	1
0	0	0
-1	-2	-1

*

=

(滤波器：人工设计)

feature map
(特征映射层)

0			

注：* 为卷积操作

$$F[0,0] = 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0$$



建立模型

卷积运算

input image

10	10	10	10	10	10
10	10	10	10	10	10
10	10	10	10	10	10
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

kernel

1	2	1
0	0	0
-1	-2	-1

*

=

feature map

0	0		

$$F[0,0] = 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0$$

$$F[0,1] = 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0$$



建立模型

卷积运算

input image

10	10	10	10	10	10
10	10	10	10	10	10
10	10	10	10	10	10
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

kernel

1	2	1
0	0	0
-1	-2	-1

*

=

feature map

0	0	0	0
40			

$$F[0,0] = 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0$$

$$F[0,1] = 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0$$

$$F[0,2] = 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0$$

$$F[0,3] = 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0$$

$$F[1,0] = 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 0 * (-1) + 0 * (-2) + 0 * (-1) = 40$$

卷积运算

对于卷积神经网络：
卷积核中的参数需要在训练的过程学习

input image

10	10	10	10	10	10
10	10	10	10	10	10
10	10	10	10	10	10
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

*

kernel

1	2	1
0	0	0
-1	-2	-1

=

feature map

0	0	0	0
40	40	40	40
40	40	40	40
0	0	0	0

$$\begin{aligned}
 F[0,0] &= 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0 \\
 F[0,1] &= 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0 \\
 F[0,2] &= 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0 \\
 F[0,3] &= 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 10 * (-1) + 10 * (-2) + 10 * (-1) = 0 \\
 F[1,0] &= 10 * 1 + 10 * 2 + 10 * 1 + 10 * 0 + 10 * 0 + 10 * 0 + 0 * (-1) + 0 * (-2) + 0 * (-1) = 40 \dots\dots \\
 F[3,3] &= 0 * 1 + 0 * 2 + 0 * 1 + 0 * 0 + 0 * 0 + 0 * 0 + 0 * (-1) + 0 * (-2) + 0 * (-1) = 0
 \end{aligned}$$



建立模型

卷积运算

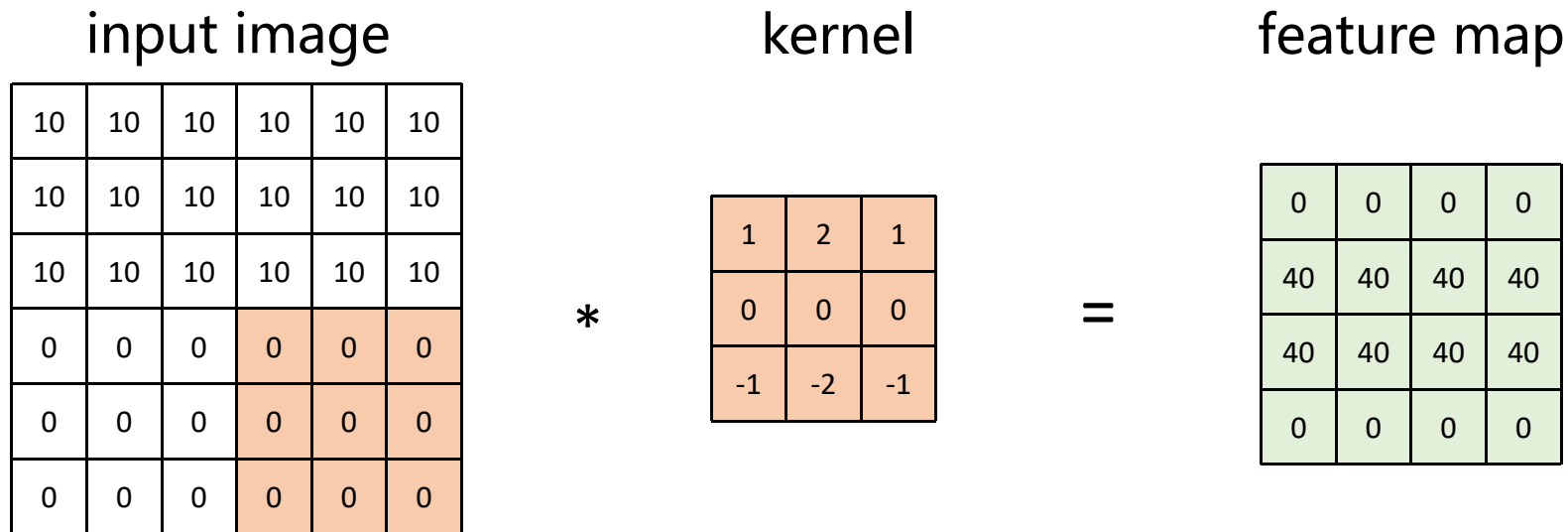


image size= $w_I \times h_I$;

kernel size= $w_K \times h_K$;

stride=1; stride为卷积窗口移动的步长

feature map size = $w_f \times h_f$,

其中 $w_f = \frac{w_I - w_K}{stride} + 1$, $h_f = \frac{h_I - h_K}{stride} + 1$



建立模型

卷积运算

1	2	3
4	5	6
7	8	9

*

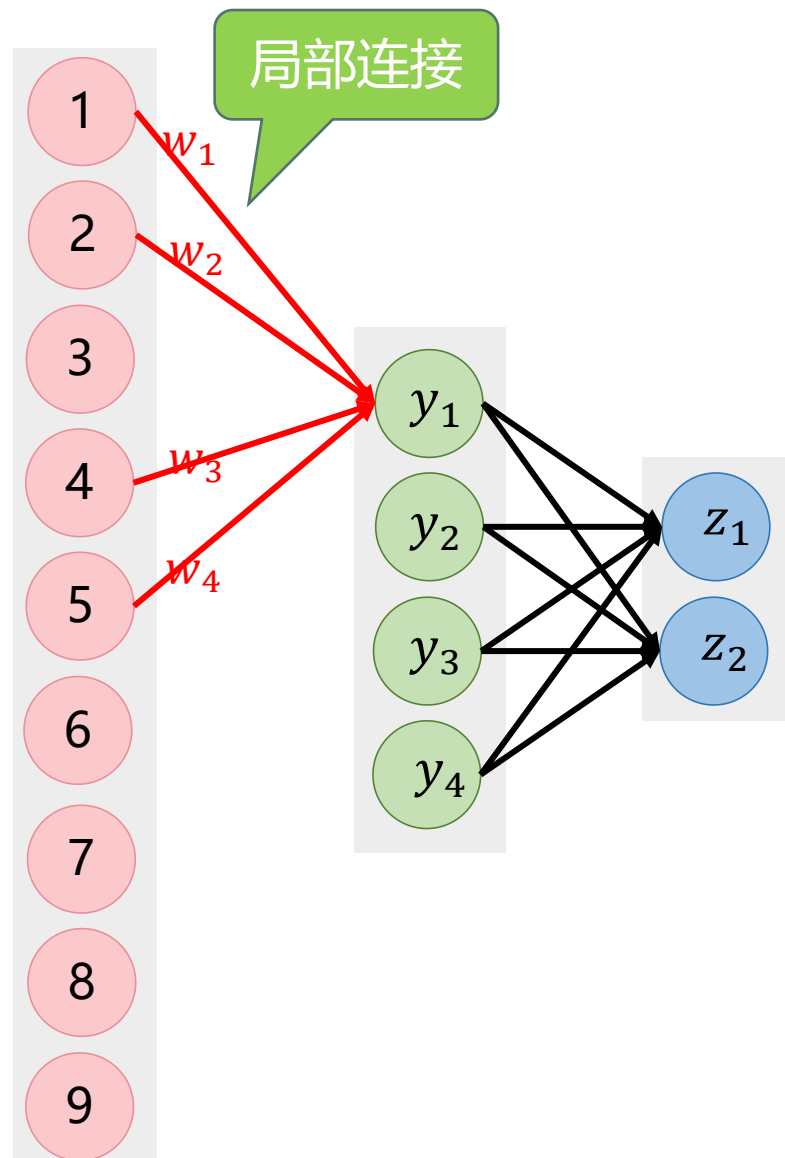
kernel

w_1	w_2
w_3	w_4

=

feature map

A	B
C	D





建立模型

卷积运算

1	2	3
4	5	6
7	8	9

*

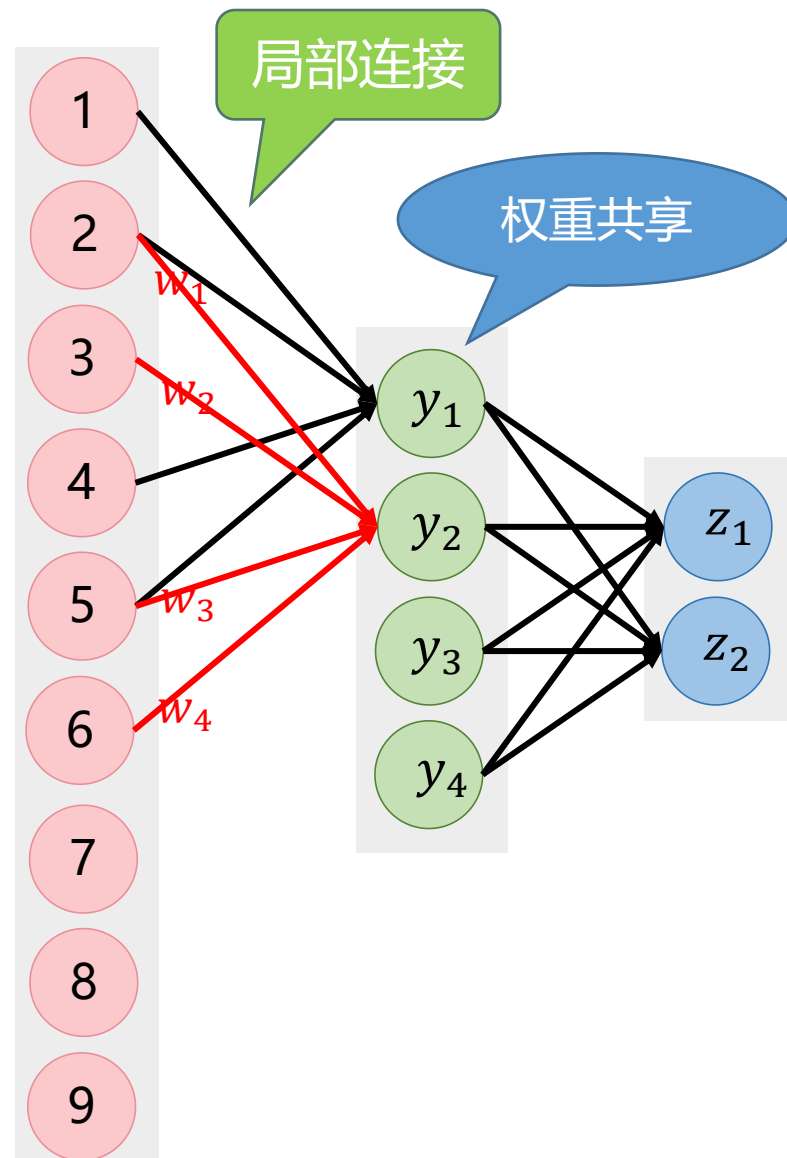
kernel

w_1	w_2
w_3	w_4

=

feature map

y_1	y_2





建立模型

卷积运算

1	2	3
4	5	6
7	8	9

*

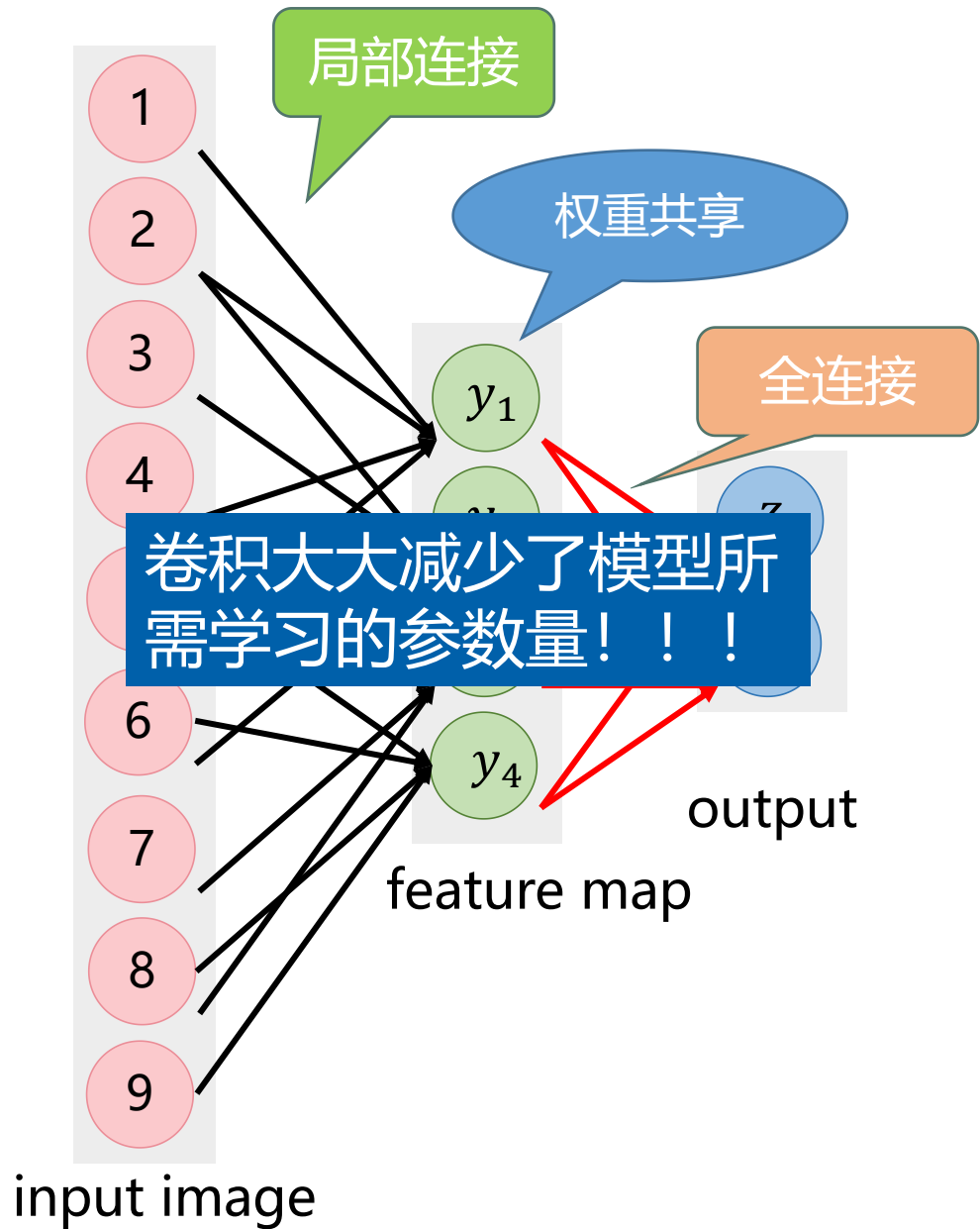
kernel

w_1	w_2
w_3	w_4

=

feature map

y_1	y_2
y_3	y_4





建立模型

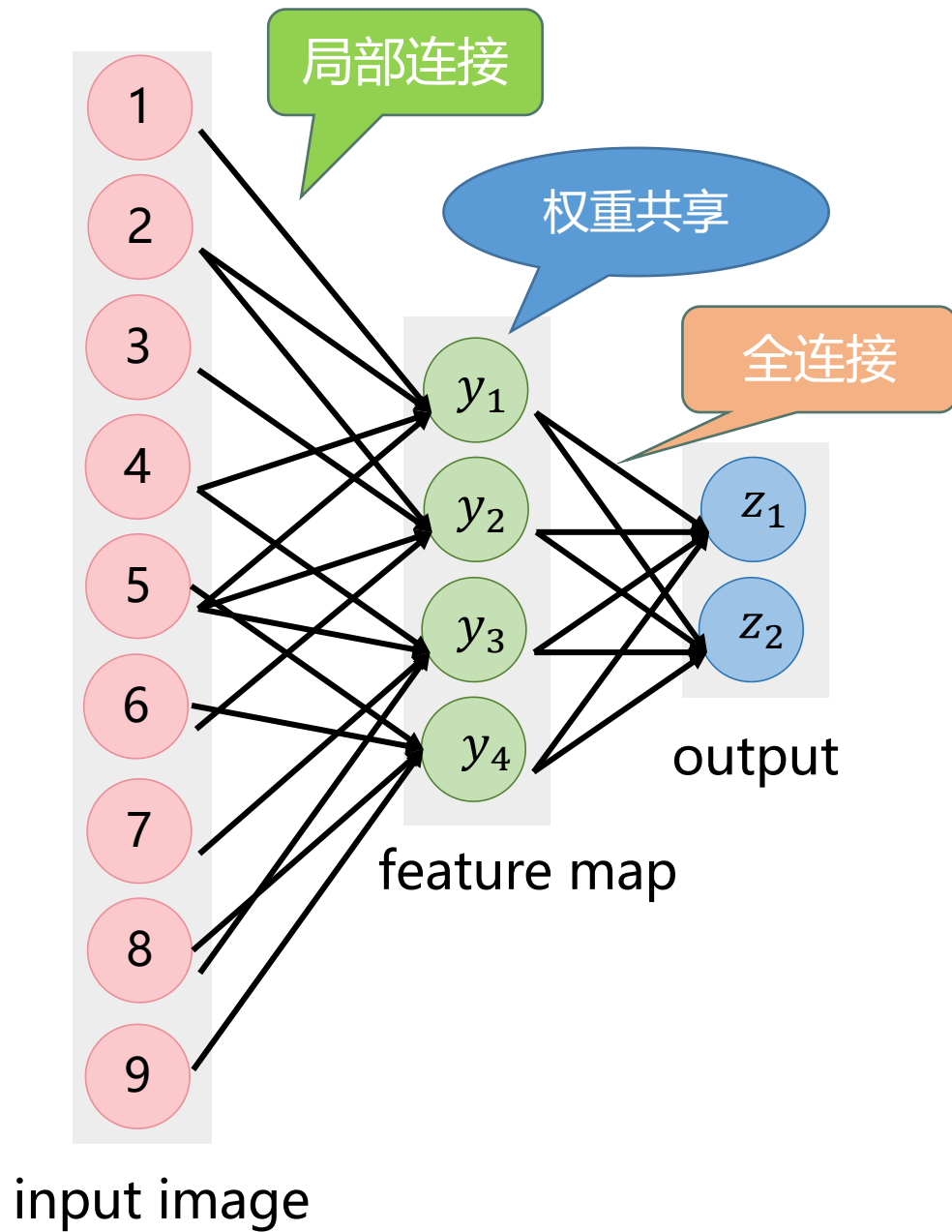
卷积运算

$$\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} * \begin{array}{|c|c|} \hline w_1 & w_2 \\ \hline w_3 & w_4 \\ \hline \end{array} = \begin{array}{|c|c|} \hline y_1 & y_2 \\ \hline y_3 & y_4 \\ \hline \end{array}$$

kernel feature map

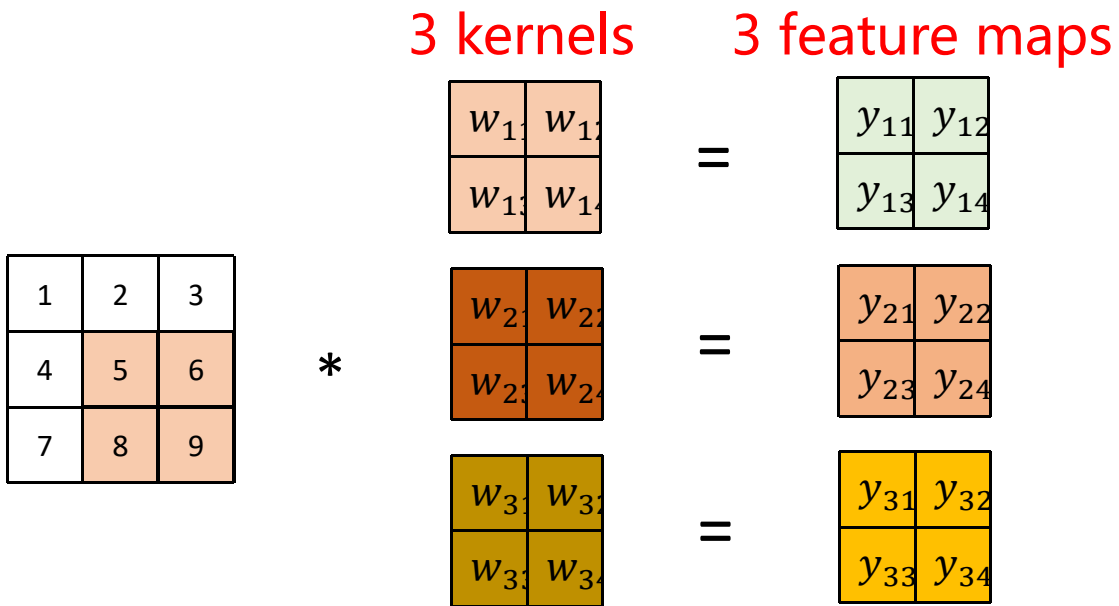


单个卷积核只能提取单一特征，如何利用卷积核提取更复杂的特征？

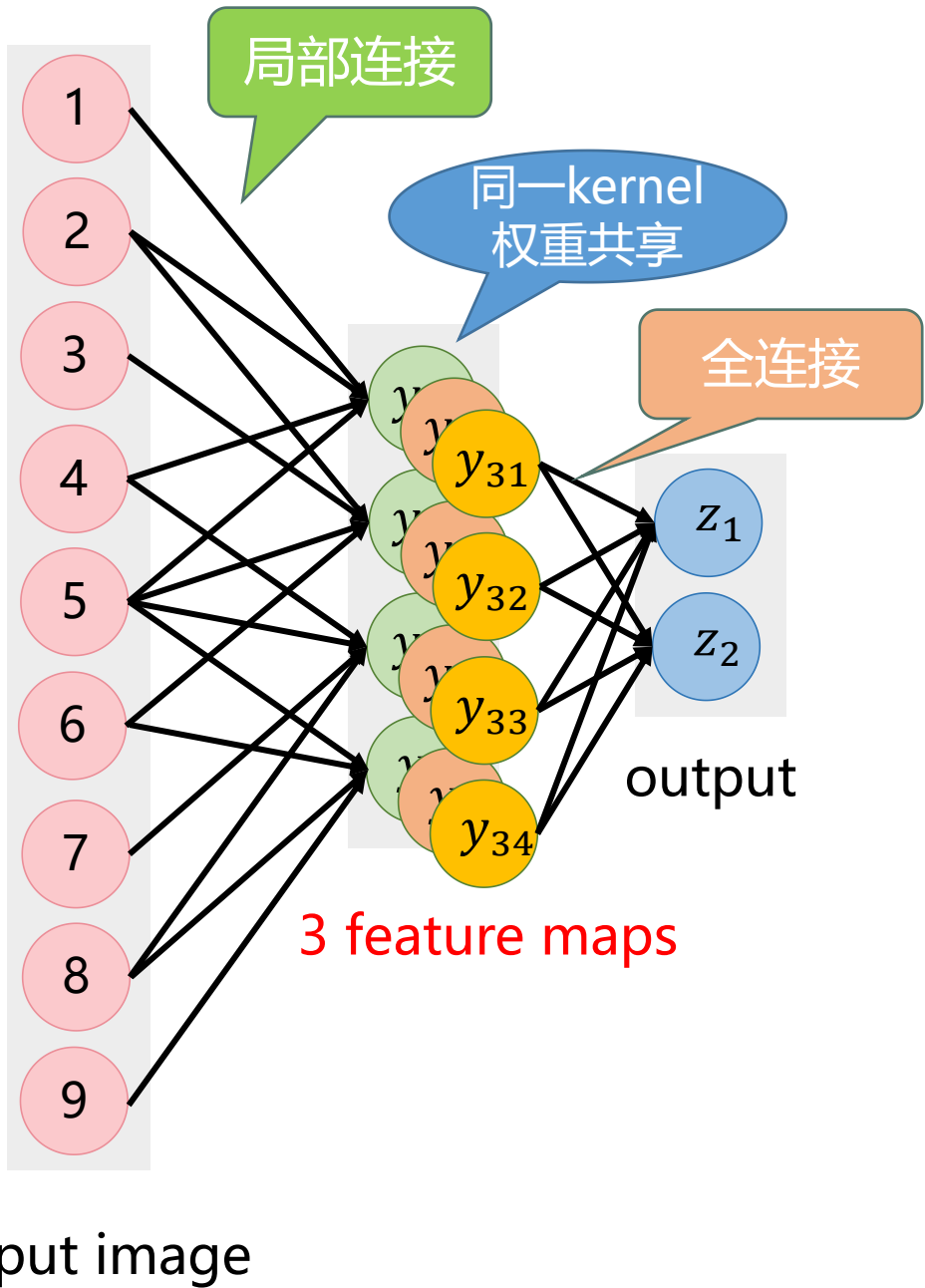


建立模型

卷积运算



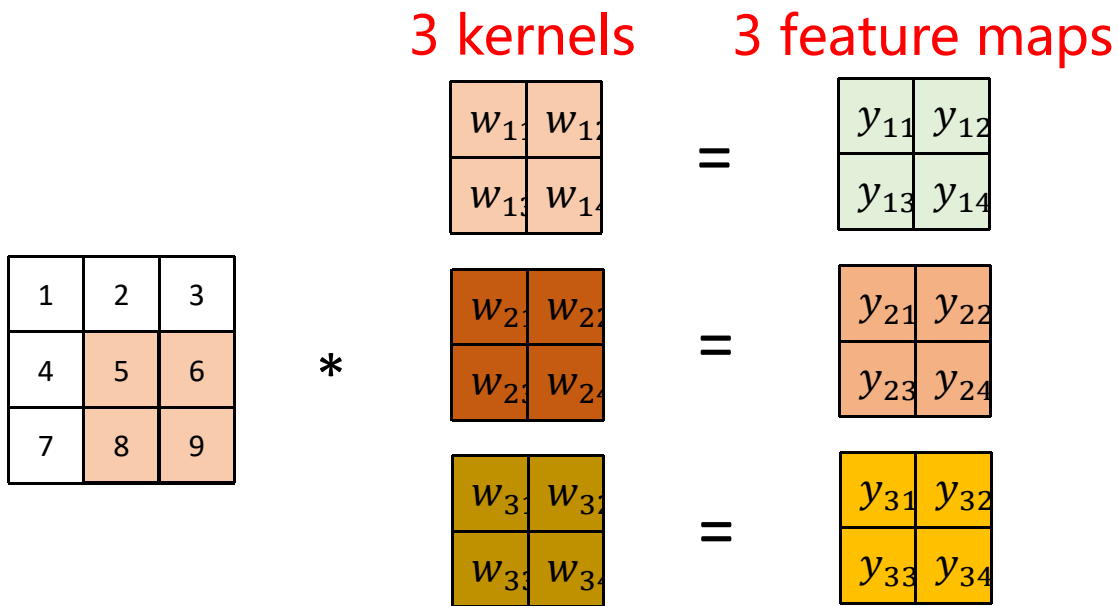
一个卷积核可以提取图像的一种特征
 多个卷积核提取多种特征。



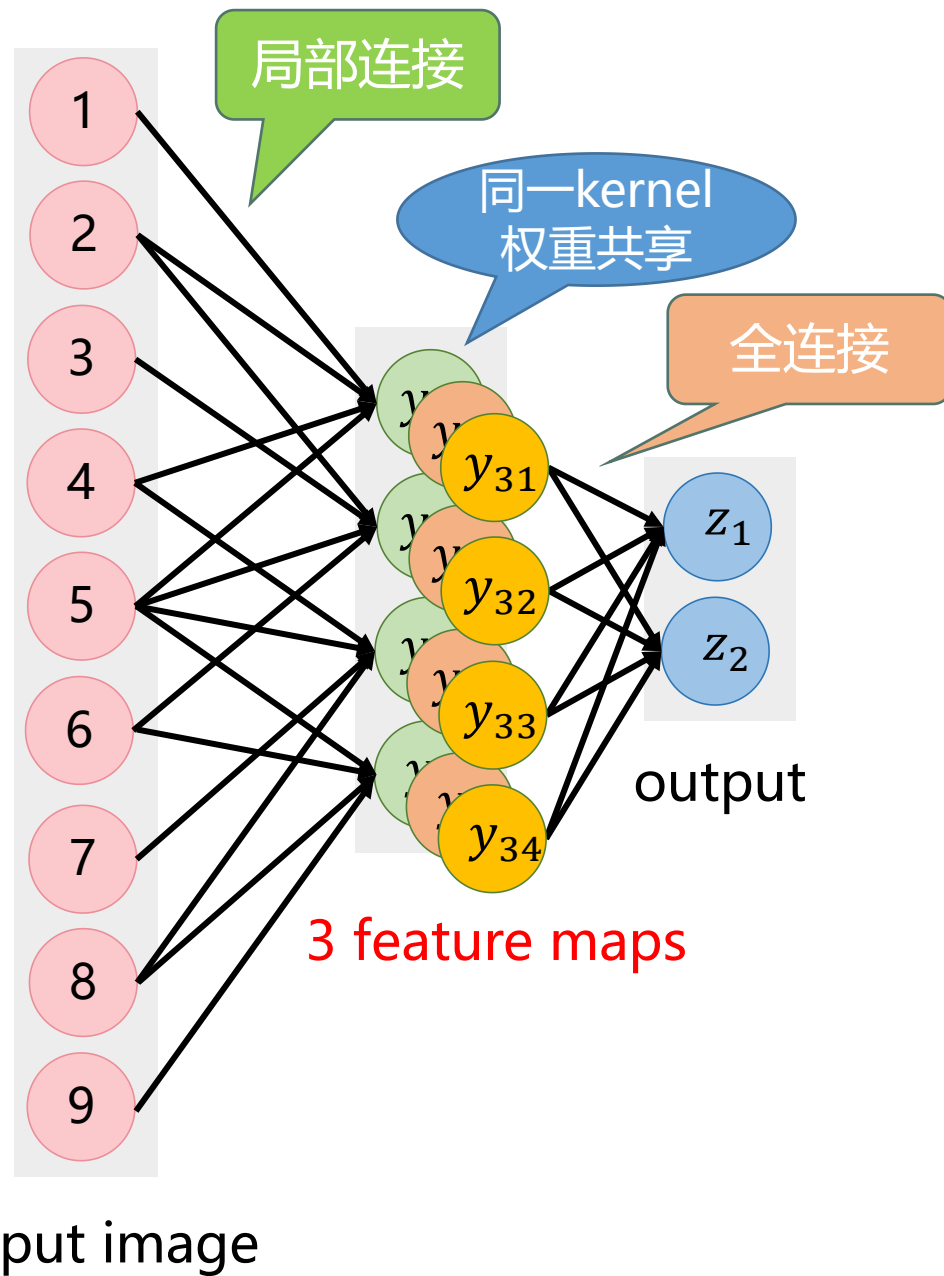


建立模型

卷积运算



对于如上size为3*3的image，如需提取100个特征，卷积层需要100个卷积核，假设卷积核大小为4，则共需4*100个参数。





建立模型

卷积运算

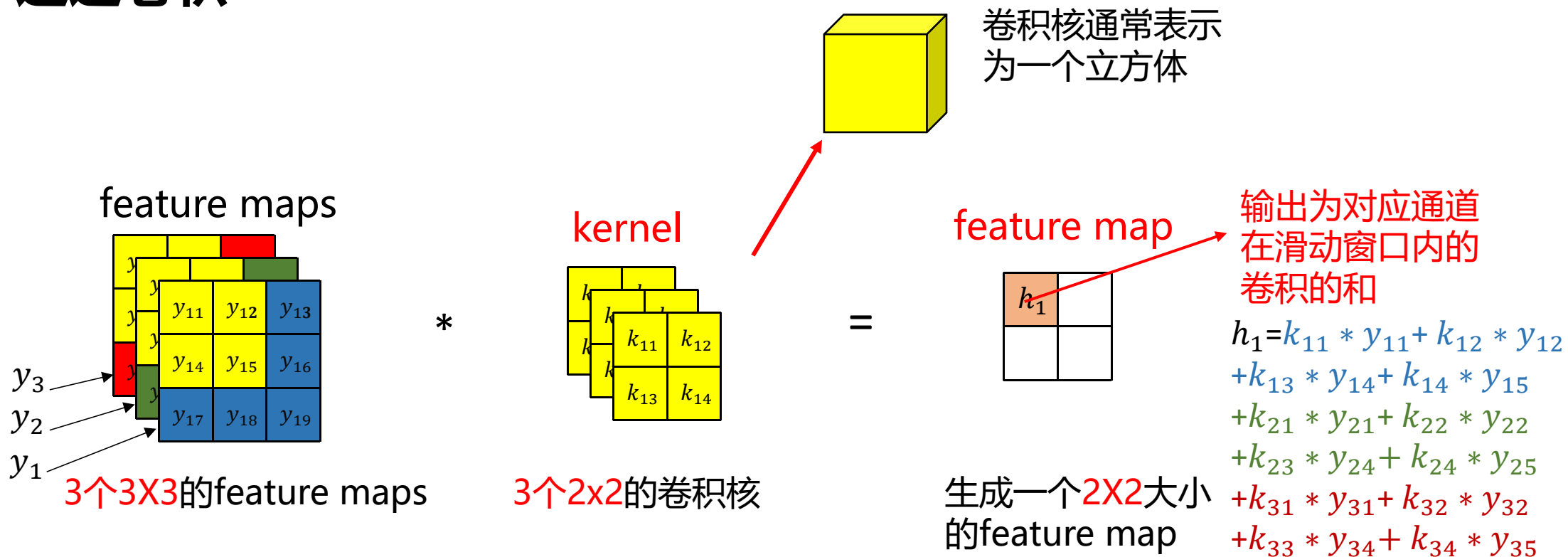
如何对feature maps继续
继续进行卷积操作？





建立模型

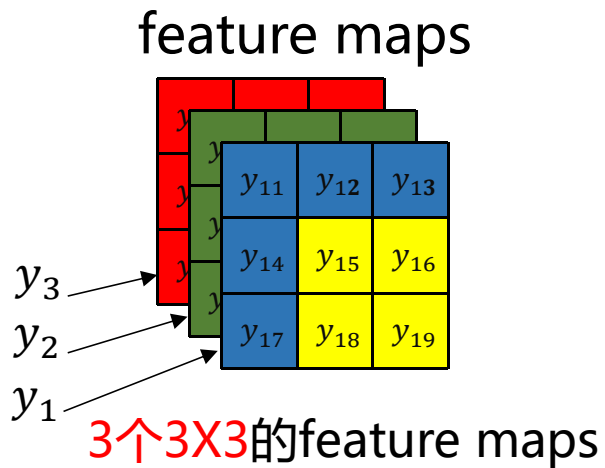
多通道卷积



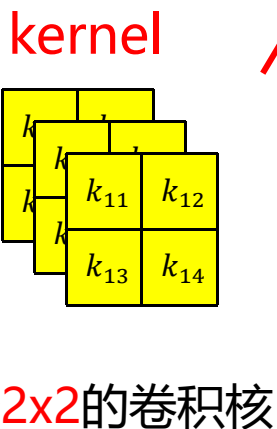


建立模型

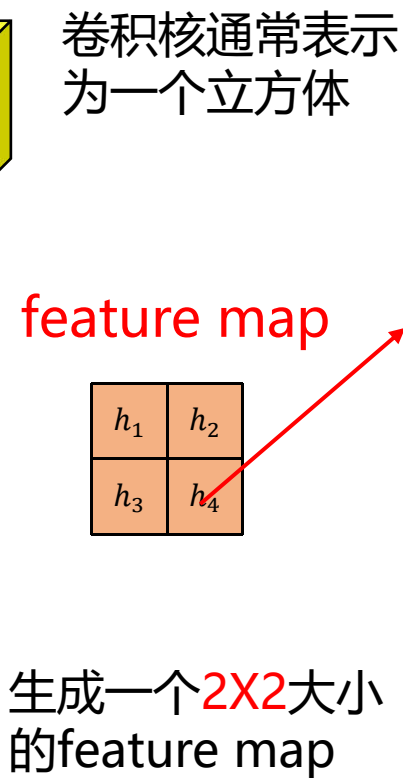
多通道卷积



*



=



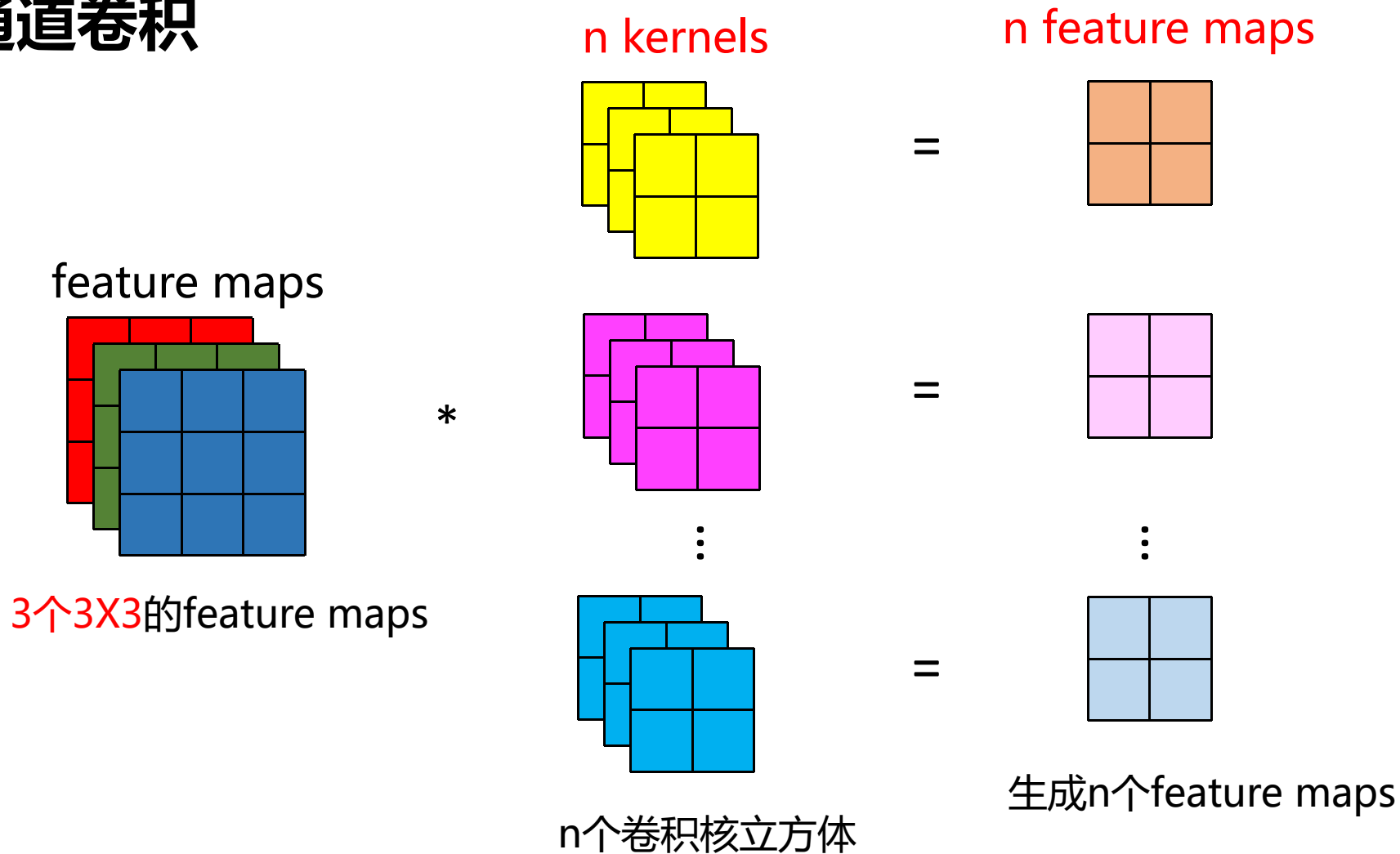
输出为对应通道在滑动窗口内的卷积的和

$$h_4 = k_{11} * y_{15} + k_{12} * y_{16} + k_{13} * y_{18} + k_{14} * y_{19} + k_{21} * y_{25} + k_{22} * y_{26} + k_{23} * y_{28} + k_{24} * y_{29} + k_{31} * y_{35} + k_{32} * y_{36} + k_{33} * y_{38} + k_{34} * y_{39} + \dots$$



建立模型

多通道卷积





建立模型

损失函数设计和参数学习的基本原则与全连接神经网络相同

池化 (Pooling)

通过下采样缩减feature map尺度，常用max pooling和average pooling

10	10	10	8	10	9
10	10	10	10	10	10
10	10	10	10	7	10
0	0	0	10	0	0
0	0	0	10	0	0
0	0	0	0	0	0

*

kernel

1	2	1
0	0	0
-1	-2	-1

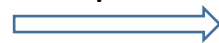
=

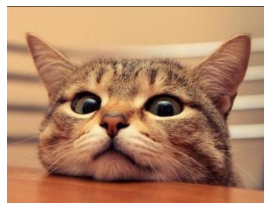
feature map

0	-2	-1	5
40	30	20	30
40	30	17	24
0	10	20	10

下采样后的
feature map

Max pooling





640x480

下采样



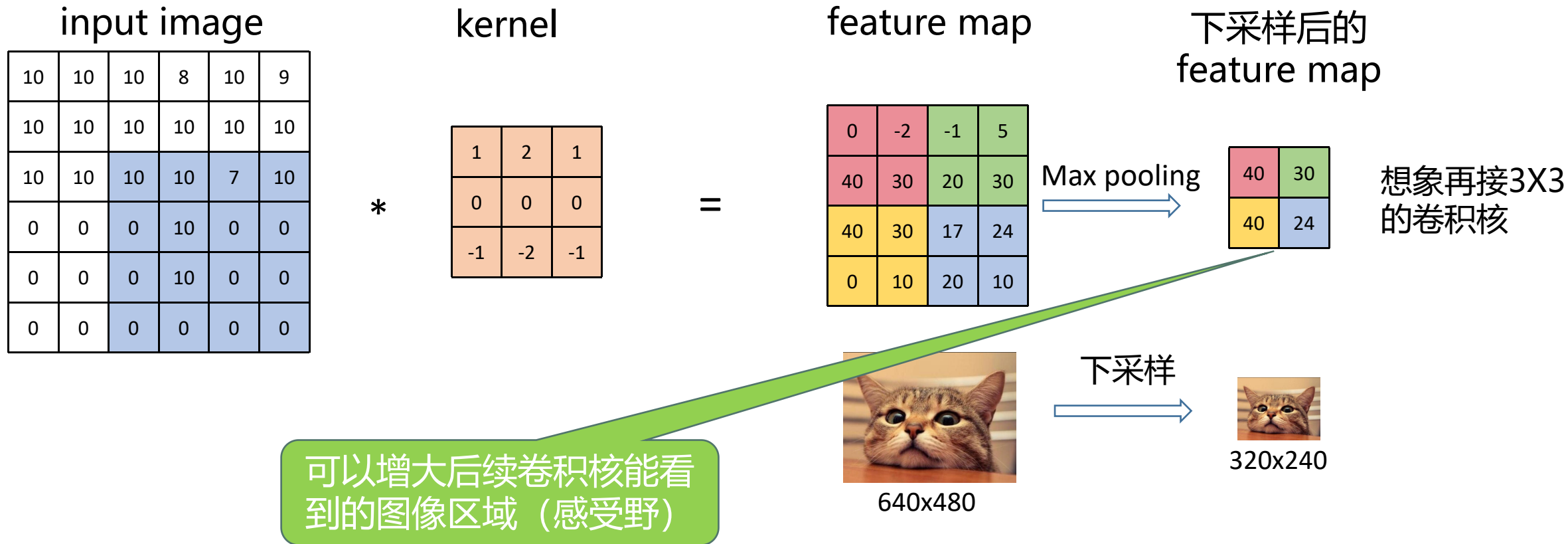
320x240



建立模型

池化 (Pooling)

通过下采样缩减feature map尺度，常用max pooling和average pooling

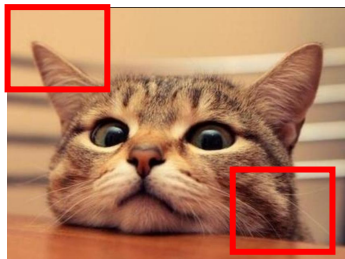


Part 4

Transformer基础

Transformer引入

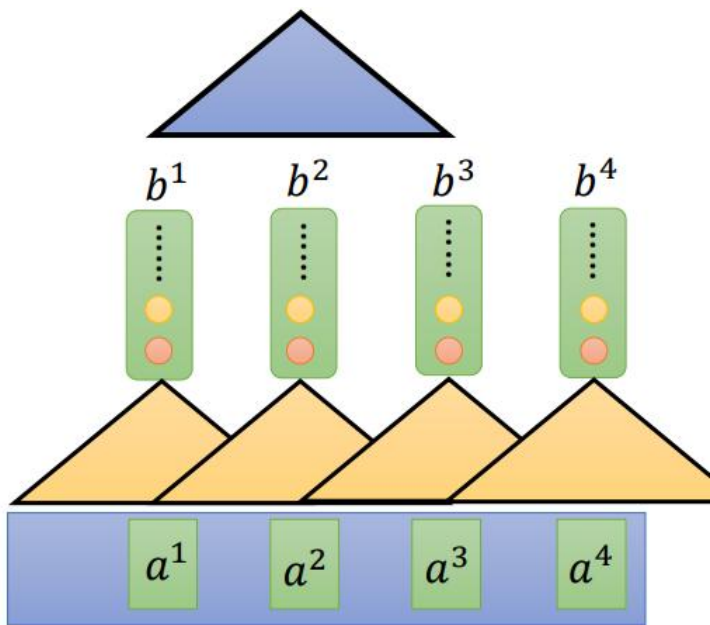
注意力：发现并关注待处理数据中的关键信息



640x480

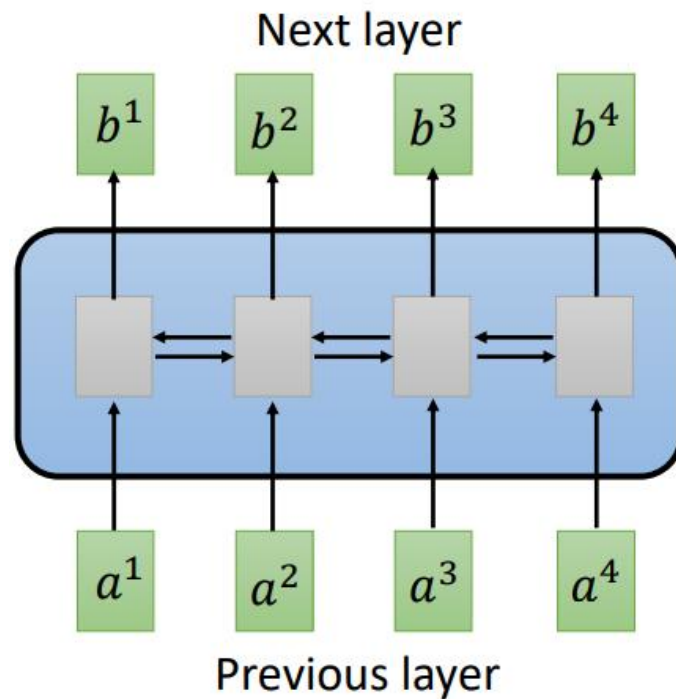
首先需要知道数据中不同部分的相关性

卷积运算可以建立单个红框内像素的关联，但对于左上和右下两个红框中的像素，关联它们的代价很高



方案1：卷积神经网络经历多个卷积-池化层，特征图空间分辨率逐步下降，同一个卷积核终于可以同时看到它们了！

网络需要很深，计算代价高！



方案2：将不同框视为一个连续序列，进而采用一种叫循环网络的模型建立它们的关联。

隔得太远了，容易忘记之前的内容；一步步串行建模，速度慢



Transformer引入

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

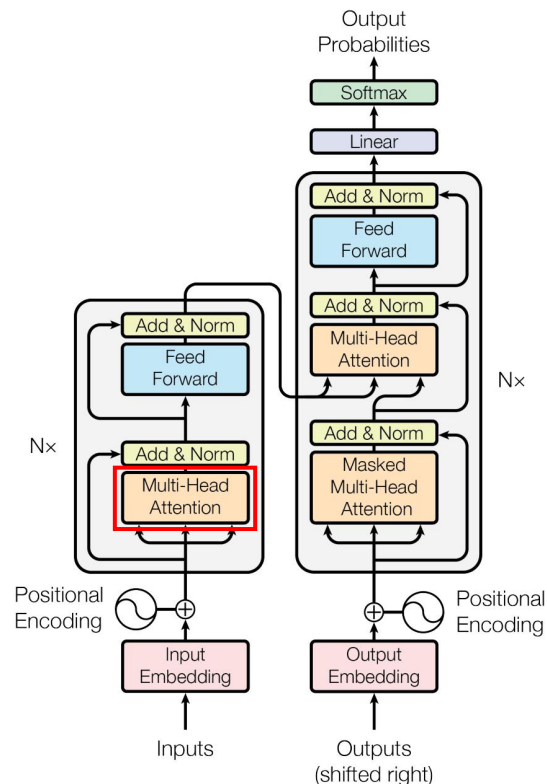
Attention is all you need

A Vaswani, N Shazeer, N Parmar... - Advances in neural ..., 2017 - proceedings.neurips.cc

... to attend to **all** positions in the decoder up to and including that position. **We need** to prevent

... **We** implement this inside of scaled dot-product **attention** by masking out (setting to $-\infty$) ...

☆ Save 📄 Cite Cited by 269221 Related articles All 26 versions 🔗



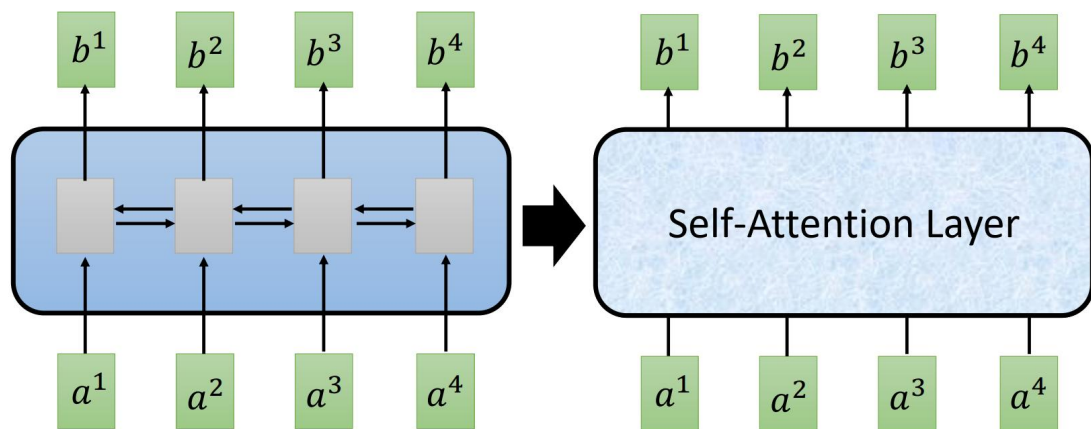
把各种模态的数据首先表示成序列形式，然后通过左边这样一个结构（左半部分、多次利用）计算序列中任意Token（词元）的相关性



相关性可视化

Transformer基础

出发点：设计一个方法，能够快速计算序列中词元的相关性

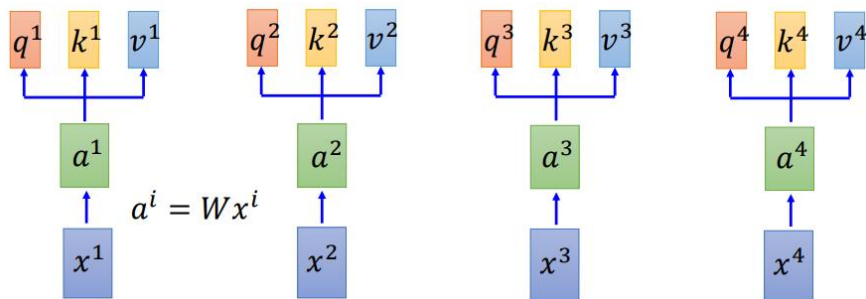
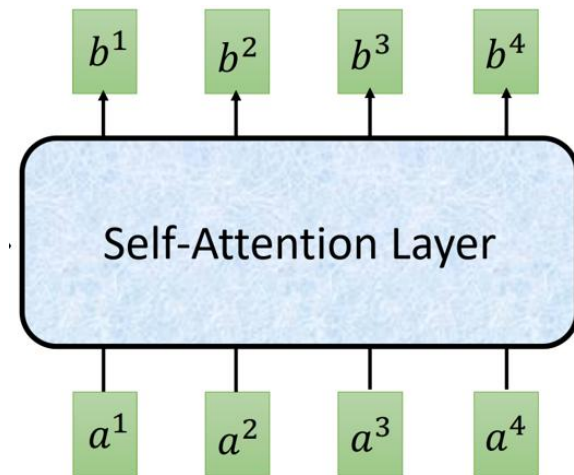


- **目标明确**：将左边的串行实现改造成右边的并行实现
- **方法新颖**：论文作者设计了一套“近似于看不懂”的方法来实现这个目标
- **效果好**：若不是效果好，可能就成为遗珠了



Transformer基础

计算过程



q : query (to match others)

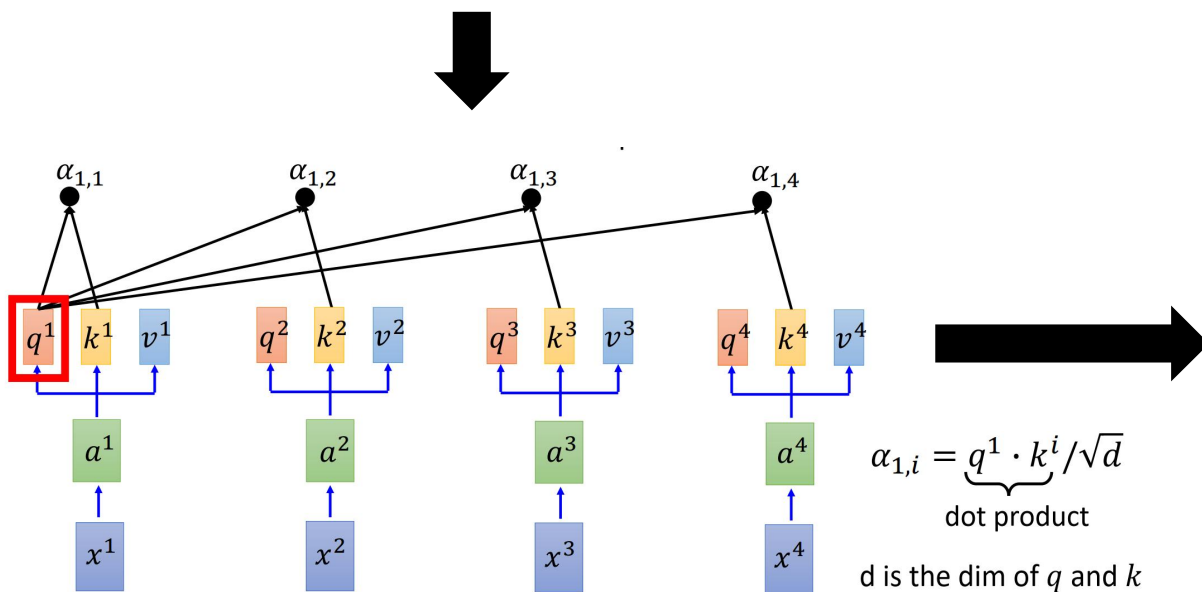
$$q^i = W^q a^i$$

k : key (to be matched)

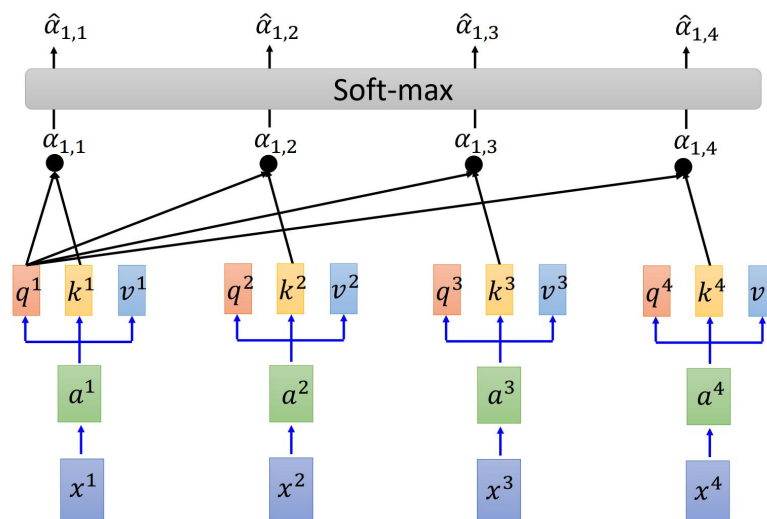
$$k^i = W^k a^i$$

v : information to be extracted

$$v^i = W^v a^i$$



$$\hat{a}_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$





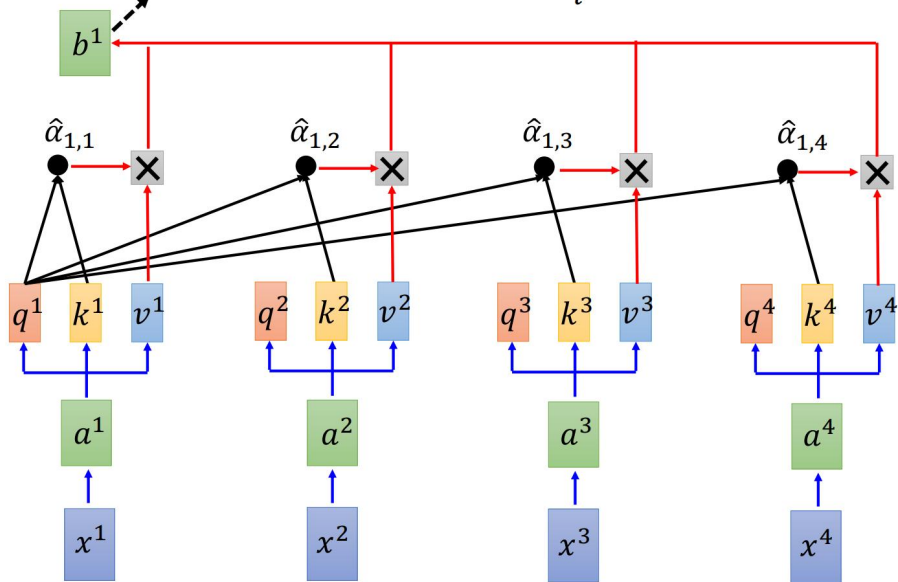
Transformer基础

计算过程

Self-attention

Considering the whole sequence

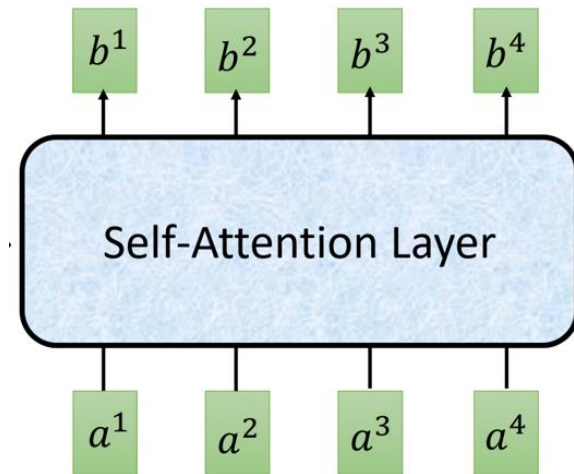
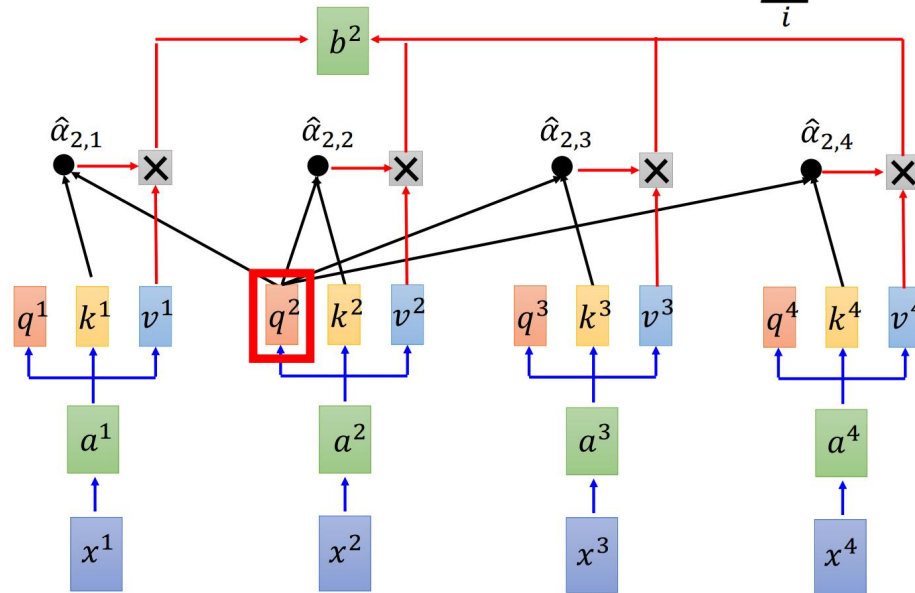
$$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$$



Self-attention

拿每个 query q 去对每个 key k 做 attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$





Transformer基础

计算过程

Self-attention

$$q^i = W^q a^i$$

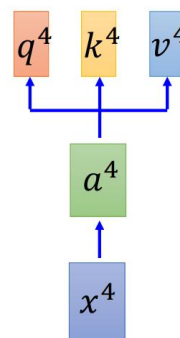
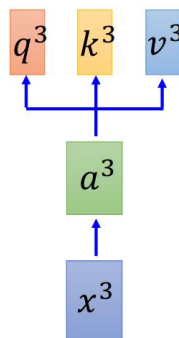
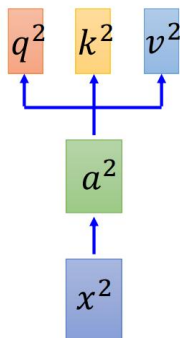
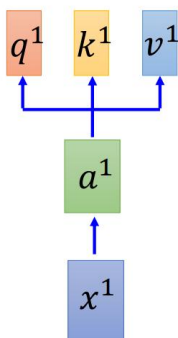
$$k^i = W^k a^i$$

$$v^i = W^v a^i$$

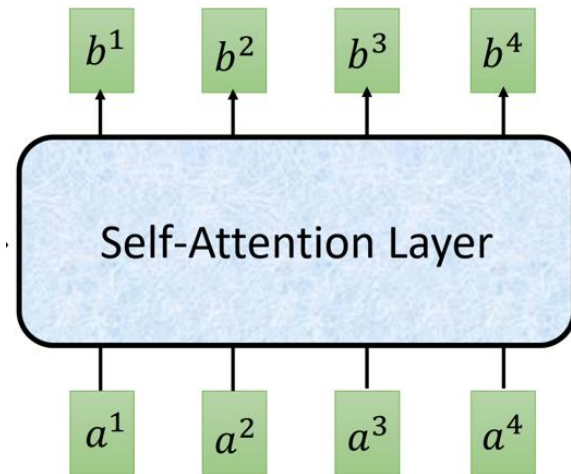
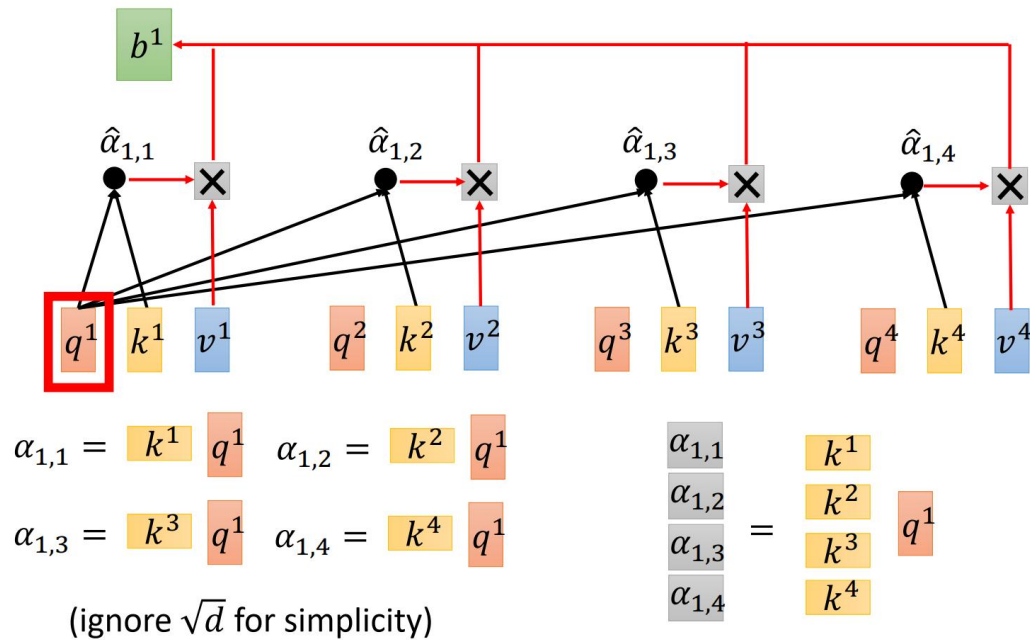
$$\begin{matrix} q^1 & q^2 & q^3 & q^4 \\ \hline Q \end{matrix} = W^q \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix}$$

$$\begin{matrix} k^1 & k^2 & k^3 & k^4 \\ \hline K \end{matrix} = W^k \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix}$$

$$\begin{matrix} v^1 & v^2 & v^3 & v^4 \\ \hline V \end{matrix} = W^v \begin{matrix} a^1 & a^2 & a^3 & a^4 \\ \hline I \end{matrix}$$



Self-attention



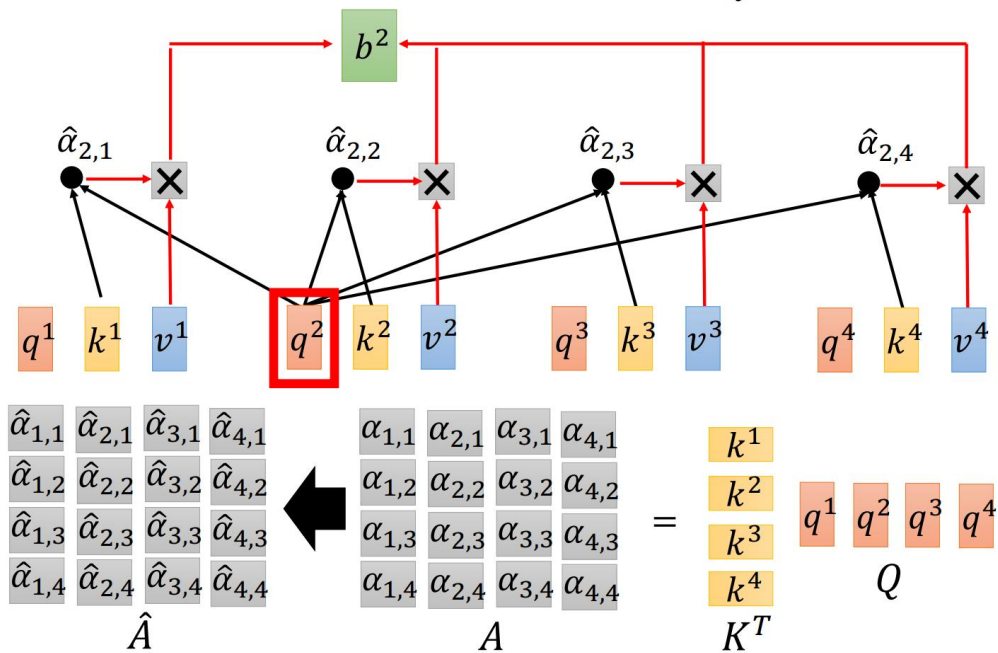


Transformer基础

计算过程

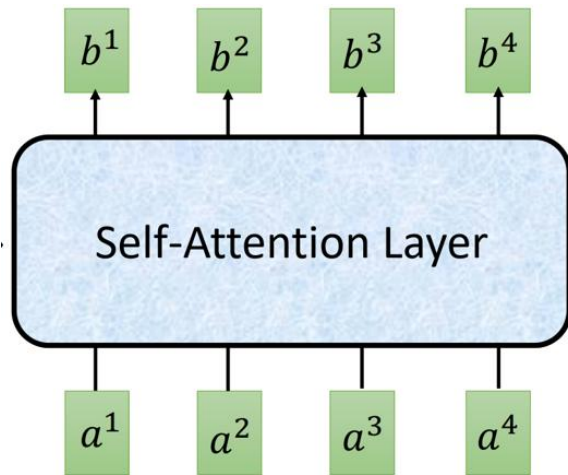
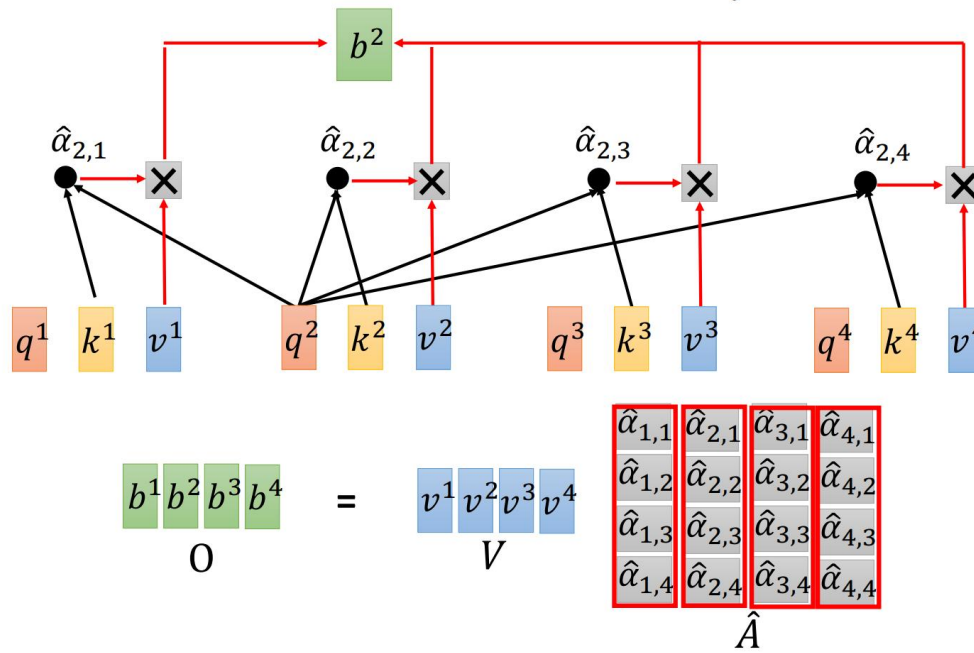
Self-attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



Self-attention

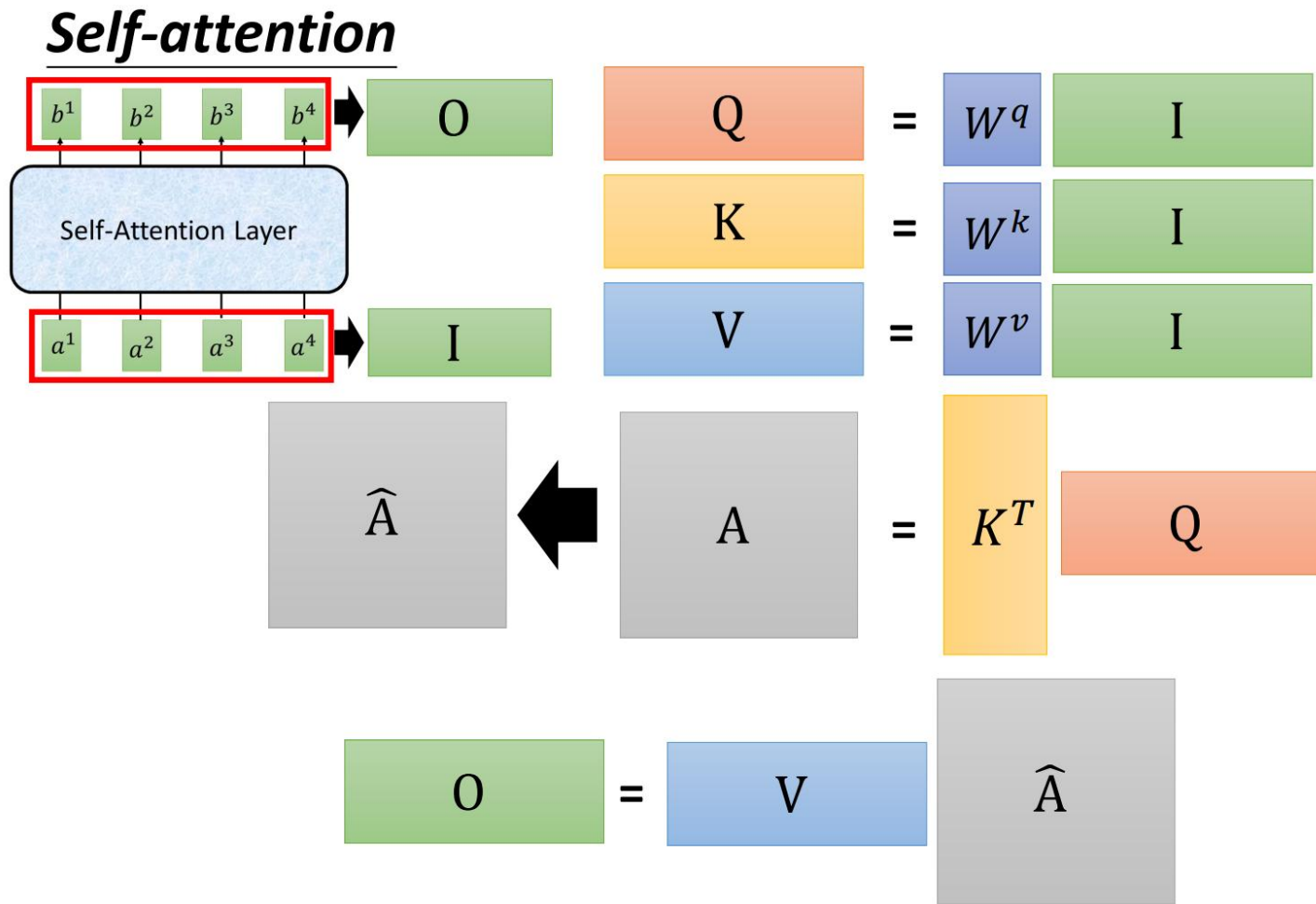
$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$





Transformer基础

计算过程



总之就是一堆矩阵乘法，用GPU可以加速

另一种解释：

Q：问题；K：线索；V：答案
通过问题本身+相关的线索，去找答案

QKV一样来源是自注意力；Q和KV不一样是交叉注意力



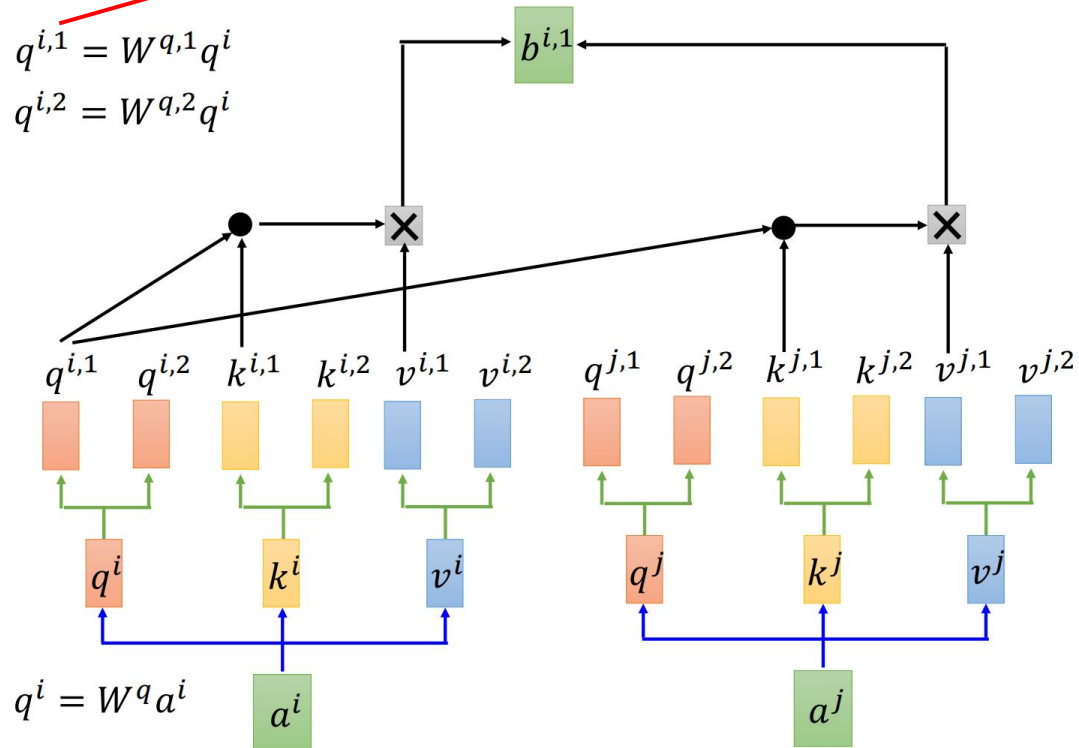
Transformer基础

多头注意力

把一个向量切成几段，例如
512维的向量切成4段128维的

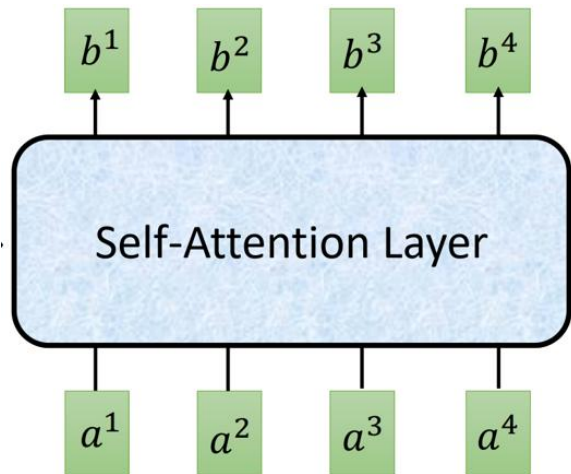
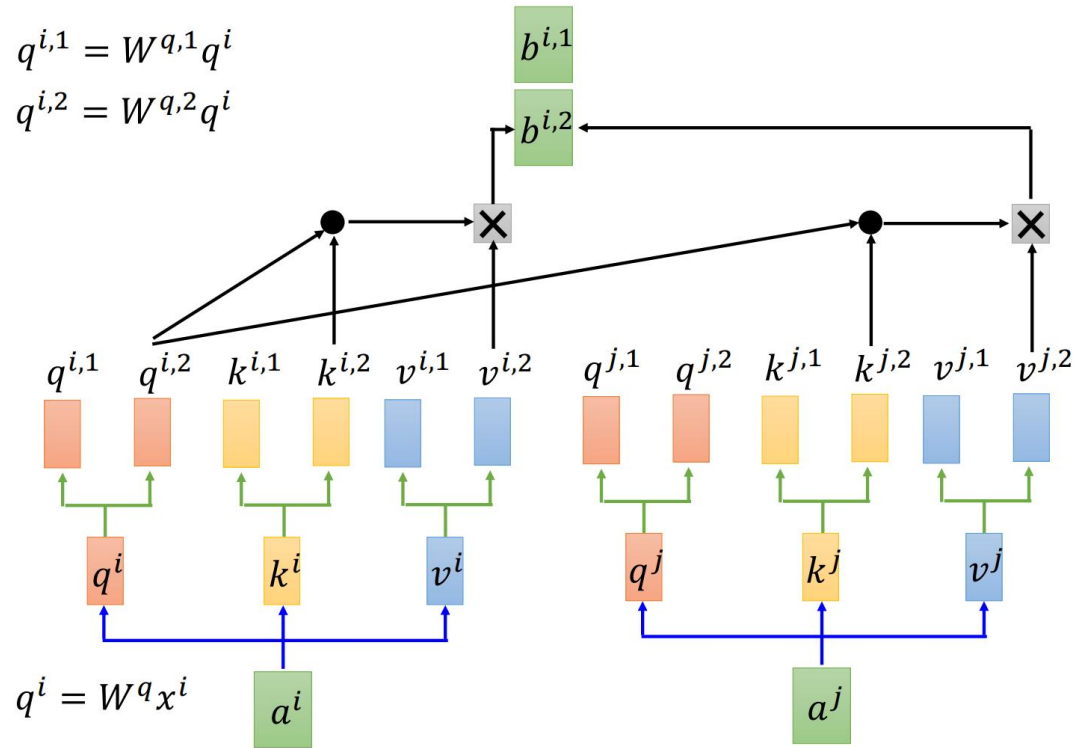
Multi-head Self-attention

(2 heads as example)



Multi-head Self-attention

(2 heads as example)

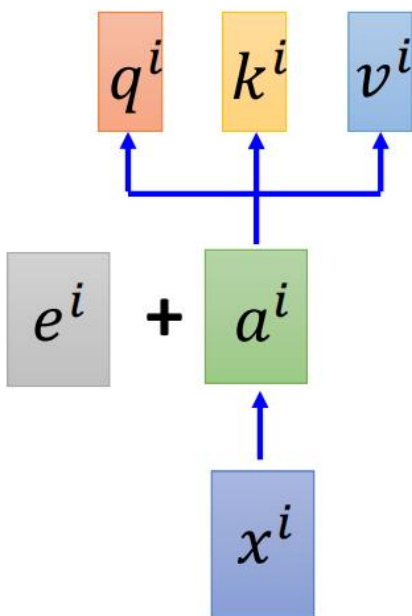
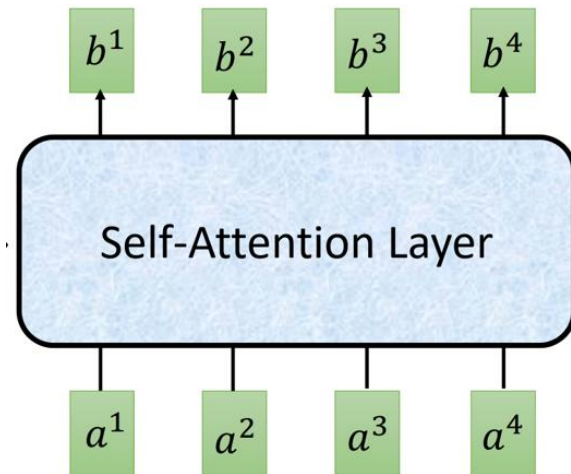




Transformer基础

位置编码

Transformer的自注意计算中，各个 a^i 没有位置信息
会有什么问题，怎么办？



$$\begin{matrix} \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \end{matrix} p^i = \begin{matrix} \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \end{matrix} \leftarrow \text{i-th dim}$$
$$\begin{bmatrix} W^I & W^P \end{bmatrix} \begin{bmatrix} x^i \\ p^i \end{bmatrix} = \begin{bmatrix} W^I x^i \\ W^P p^i \end{bmatrix} + e^i = a^i$$

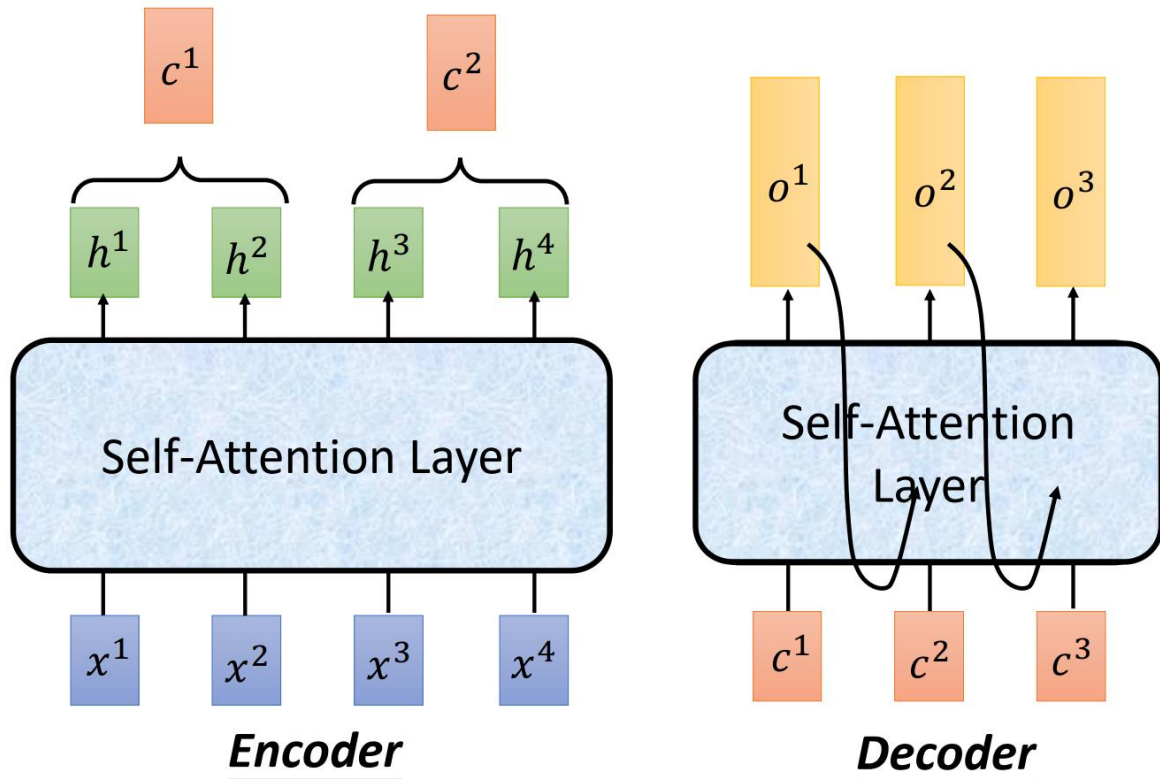
序列生成时加上去，后面的计算都不变



Transformer基础

Transformer的使用

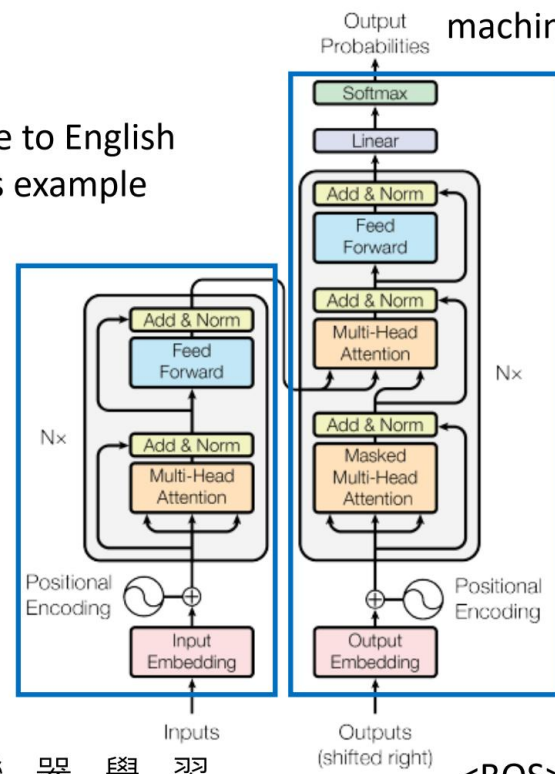
Seq2seq with Attention



Transformer

Using Chinese to English translation as example

Encoder



机器学习

Output Probabilities machine learning

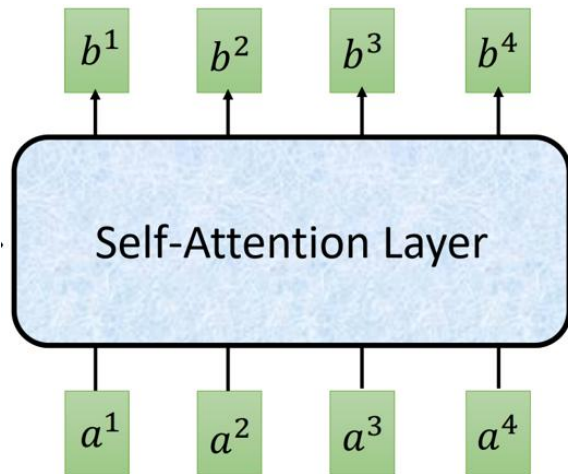
机器翻译示例

Decoder

Outputs (shifted right)

<BOS>

machine





Transformer 的解码：输出是怎样生成的

把“训练时可并行”与“推理时如何出字”分开看

自回归解码 (AR)

输入：我 喜欢
第 1 步预测 “学习”

输入：我 喜欢 学习
第 2 步预测 “Transformer”

每一步依赖前一步，质量强，速度受限

并行 / 非自回归解码

一次预测多个位置
[我] [喜欢] [学习] [Transformer]

可同时修正整句
适合固定长度或迭代式生成

吞吐更高，通常需要额外训练目标或迭代



并行解码：一次预测所有位置

速度来自条件独立假设，质量问题来自位置之间缺少直接依赖

$$p(y \mid x) = \prod_i p(y_i \mid x)$$

输入表示
Encoder(x)



预测目标长度 L
建立 L 个位置



并行预测
 $y_1, y_2, \dots, y_L$

为什么快

所有位置一次前向计算
不等待前一个 token

为什么会错

每个位置各自做决定
容易重复、漏词或混合不同表达

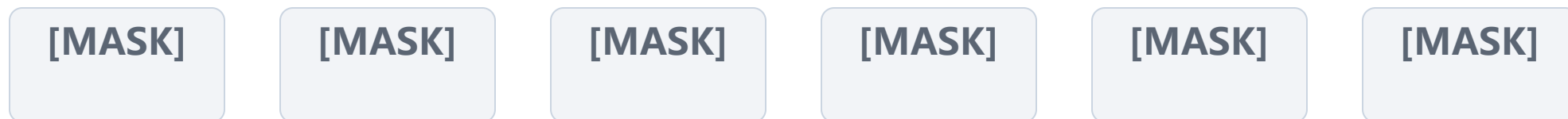
常见补救：先并行生成草稿，再遮住低置信位置并迭代重写



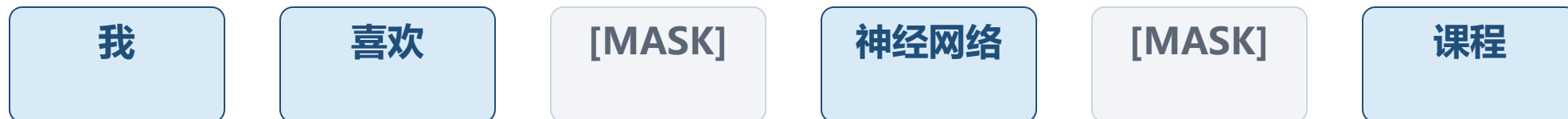
Mask diffusion: 先留空, 再逐步补全

把“从左到右写句子”改成“反复修正整句”

初始化: 整句都是未知



迭代去噪: 每轮同时预测一批 token, 保留高置信结果



优点: 全局修订、天然并行; 代价: 需要多轮迭代, 训练与采样方式更复杂



Mask diffusion: 随机加噪与逐步去噪

训练时学习恢复任意遮盖程度，生成时从全 MASK 反向恢复

前向加噪 $q(x_n | x_0)$: 按噪声强度 t 随机把 token 替换为 [MASK]



反向去噪 $p_\theta(x_{n-1} | x_n, t)$: 模型预测被遮盖 token, 并选择一部分位置恢复



与普通 Mask-Predict 的区别: 扩散模型显式采样噪声时刻 t , 并按噪声日程学习整条去噪过程

THANKS